

UNIVERSIDAD DE LA HABANA
FACULTAD DE MATEMÁTICA Y COMPUTACIÓN

Un compilador para

HULK

*Diseño, implementación y extensión de un
lenguaje de programación orientado a objetos*

REPORTE DE PROYECTO FINAL
Asignaturas de Compilación y Lenguajes de Programación

CURSO	2025–2026
CARRERA	Ciencias de la Computación
AÑO	Tercer año
BACKEND	LLVM 17 (código nativo x86-64)
STACK	RUST 1.77+
EXTENSIÓN	Vectores y expresiones generadoras

Resumen

SE presenta un compilador *ahead of time* para HULK (*Havana University Language for Compilers*), un lenguaje multiparadigma estáticamente tipado, orientado a expresiones y con un sistema de objetos basado en herencia simple y protocolos estructurales. El compilador está escrito íntegramente en RUST (edición 2024) y produce ejecutables nativos x86-64 mediante LLVM 17, accedido a través de la biblioteca *inkwell*. El *frontend* (analizadores léxico, sintáctico y semántico) y el *backend* (generación de LLVM IR, optimización y enlace con un *runtime* en C) se implementan sin recurrir a generadores externos de *lexers* ni *parsers*: el analizador léxico se construye sobre autómatas finitos no deterministas obtenidos por la construcción clásica de Thompson, y el analizador sintáctico es un motor LALR(1) cuyas tablas se generan en tiempo de inicialización a partir de una definición declarativa de la gramática de HULK.

La implementación cubre todas las características nucleares del lenguaje —expresiones aritméticas y de cadena, control de flujo expresivo, funciones de primer orden, tipos con herencia simple, protocolos con conformidad estructural y los operadores de prueba y conversión de tipos *is/as*—, e incorpora como aporte propio un **sistema de vectores con expresiones generadoras**. Esta extensión añade un constructor de tipos paramétrico `Vector<T>` al sistema de tipos, tres nuevas formas sintácticas (literales, generadores e indexación) y una semántica operacional ansiosa con verificación de límites en tiempo de ejecución. Su diseño está inscrito en la familia clásica de las *list comprehensions* de Haskell [7] y Python, las *for-comprehensions* de Scala [8] y los *ranges and views* de C++20, y se compara con cada una de ellas para justificar las decisiones de diseño tomadas.

El cuerpo del documento analiza con detalle las cinco fases del compilador, expone las decisiones técnicas relevantes junto a sus alternativas descartadas, formaliza la sintaxis y la semántica del lenguaje implementado mediante reglas de inferencia tipográfica y semántica operacional de gran paso, y reporta los resultados de validación obtenidos al ejecutar la suite de pruebas sobre el binario producido.

Índice general

Resumen	1
1 Introducción	4
1.1 El lenguaje HULK	4
1.2 Principios de diseño del compilador	5
2 Arquitectura del compilador	7
2.1 El pipeline de compilación	7

2.2	Las cuatro fases del <i>frontend</i>	8
2.3	Las fases del <i>backend</i>	9
2.4	Elección del lenguaje anfitrión: RUST	10
3	Análisis léxico	11
3.1	Arquitectura: lexer basado en NFA	11
3.1.1	Construcción de Thompson	11
3.1.2	Ejecución sin determinización	11
3.1.3	El MasterNFA y la regla del máximo prefijo	12
3.2	Conjunto de tokens reconocidos	12
3.3	Secuencias de escape	12
3.4	Recuperación de errores	13
4	Análisis sintáctico	14
4.1	Parser LALR(1) implementado desde cero	14
4.2	Pipeline del parser	14
4.2.1	Definición de la gramática	14
4.2.2	Calculo de FIRST y FOLLOW	15
4.2.3	Construcción de la tabla LALR	15
4.2.4	Motor de parseo	16
4.3	Jerarquía de precedencias	16
4.4	El árbol sintáctico abstracto	16
5	Análisis semántico	18
5.1	Estrategia en dos pasadas	18
5.2	Sistema de tipos	19
5.2.1	Categorías de tipo	19
5.2.2	Relación de conformidad	19
5.2.3	Unificación de tipos en condicionales: el ancestro común más cercano	20
5.3	Funciones y constantes predefinidas	20
5.4	Reporte de errores semánticos	21
6	Generación de código	22
6.1	LLVM IR mediante <i>inkwell</i>	22
6.2	Representación de valores: el enum CgValue	22
6.3	Representación de objetos en memoria	23
6.4	Tablas de métodos virtuales (vtables)	23
6.4.1	Despacho de métodos a través de un protocolo	24
6.5	Etiquetas de tipo en orden DFS	25
6.6	El operador as: downcast con verificación	25
6.7	La palabra clave base()	26
6.8	Pipeline de optimización	26
7	Runtime y compilación final	27
7.1	La biblioteca de runtime en C	27
7.2	Layout en memoria de vectores y rangos	27
7.3	Pipeline AOT completo	28
8	Extensiones del lenguaje implementadas	29

8.1	Protocolos: tipado estructural en tiempo de compilación	29
8.1.1	Motivación	29
8.1.2	Sintaxis, AST y sinónimo léxico <i>interface</i>	29
8.1.3	Sistema de tipos y conformidad estructural	31
8.1.4	Despacho dinámico en LLVM	32
8.1.5	Análisis comparativo	33
8.1.6	Decisiones de diseño	34
8.2	Iterables y rangos: el protocolo <i>Iterable</i>	35
8.2.1	Motivación	35
8.2.2	El protocolo <i>Iterable</i> y el tipo <i>Range</i>	35
8.2.3	La anotación <i>T*</i> y la transpilación del <i>for-loop</i>	36
8.2.4	Vectores como iterables	37
8.2.5	Análisis comparativo	37
8.2.6	Decisiones de diseño	38
8.3	Vectores: del literal a la alocaación dimensionada	39
8.3.1	Motivación	39
8.3.2	El tipo <i>Vector<T></i> en el sistema de tipos	39
8.3.3	Formas de construcción	39
8.3.4	Anotaciones de tipo: <i>T[]</i> y <i>T[][]</i>	41
8.3.5	Indexación, mutación y el método <i>size</i>	42
8.3.6	Representación en runtime y verificación de límites	42
8.3.7	Reglas de tipado formales	43
8.3.8	Semántica operacional	43
8.3.9	Generación de código para la forma generadora	44
8.3.10	Análisis comparativo	45
8.3.11	Decisiones de diseño	46
9	Decisiones de diseño significativas	48
9.1	LALR(1) artesanal frente a generador externo	48
9.2	LLVM como backend vs intérprete vs VM propia	48
9.3	Vtables vs tagged union vs boxed types	49
9.4	Type tags DFS pre-orden vs class objects	49
9.5	Enlace dinámico vs estático de LLVM	49
10	Evaluación y pruebas	50
10.1	Pruebas unitarias por módulo	50
10.2	Programas HULK de integración	50
10.3	Contrato de errores	51
11	Limitaciones conocidas y trabajo futuro	52
11.1	Limitaciones actuales	52
11.2	Trabajo futuro	53
12	Conclusiones	54
	Bibliografía	56

1

Introducción

LA traducción de programas escritos en un lenguaje de alto nivel a código nativo ejecutable es uno de los procesos fundamentales de la informática contemporánea y la actividad sobre la que se sostienen prácticamente todas las herramientas modernas de desarrollo de *software*. Tras cinco décadas de evolución la disciplina ha consolidado un *pipeline* de fases bien diferenciadas —análisis léxico, sintáctico y semántico, generación de código intermedio, optimización y emisión de código máquina— cuyos fundamentos teóricos están extensamente documentados en la literatura clásica [1, 14, 13], pero cuya implementación concreta para un lenguaje real exige resolver una gran cantidad de problemas prácticos no triviales: representación de tipos algebraicos en memoria, despacho de métodos en presencia de polimorfismo, propagación de tipos inferidos a través de fases independientes, integración con *backends* maduros como LLVM [4], manejo de errores con localización precisa, y un largo etcétera.

Este reporte describe **el diseño y la construcción de un compilador completo** para HULK (*Havana University Language for Kompilers*), un lenguaje multiparadigma de propósito didáctico cuya especificación combina elementos del paradigma funcional, imperativo y orientado a objetos. El compilador toma como entrada un fichero fuente `.hulk` y produce un ejecutable nativo Linux x86-64 que el sistema operativo puede cargar y ejecutar sin más dependencias externas que `libc` y `libm`. Toda la implementación está escrita en RUST salvo una pequeña biblioteca de soporte en tiempo de ejecución escrita en C, y utiliza LLVM 17 como infraestructura para la optimización y la emisión de código máquina.

1.1 El lenguaje HULK

HULK es un lenguaje estáticamente tipado, orientado a expresiones, con un sistema de objetos basado en herencia simple y protocolos estructurales. Tres rasgos lo definen como objetivo de compilación particularmente interesante.

Expresividad funcional. En HULK no existe la noción de *sentencia*: todas las construcciones del lenguaje son expresiones que producen un valor. Un `if-elif-else` no ejecuta ramas distintas con efecto de lado; *retorna* el valor de la rama elegida. Un bucle `while` retorna el valor de su última iteración. Un bloque $\{ e_1; e_2; \dots; e_n \}$ retorna e_n . Esta uniformidad obliga al verificador de tipos a calcular el tipo de cada construcción compuesta como el *ancestro común más cercano* de los tipos de sus subexpresiones, y obliga al generador de código a sintetizar nodos ϕ en cada punto de unión del grafo de flujo.

Tipado nominal con protocolos estructurales. La jerarquía de tipos definidos por el usuario sigue el modelo nominal clásico (dos tipos son iguales si tienen el mismo nombre), con herencia simple y sobrescritura de métodos. Sobre esa jerarquía se superpone un segundo mecanismo de polimorfismo basado en *protocolos*: tipos abstractos que declaran firmas de métodos sin cuerpo y que un tipo concreto *satisface implícitamente* con sólo declarar métodos compatibles —sin necesidad de mención explícita de la relación—. La conformidad estructural se rige por las reglas estándar de varianza (covarianza en retornos, contravarianza en parámetros) [6], lo que la convierte en una construcción rica desde el punto de vista del sistema de tipos.

Inferencia local de tipos. Las anotaciones de tipo son opcionales en parámetros, atributos e inicializadores de let. Cuando se omiten, el verificador de tipos debe inferir el tipo correcto mediante un análisis bidireccional: las firmas conocidas restringen los tipos hacia abajo, los usos restringen los tipos hacia arriba. La inferencia debe operar también en presencia de *recursión mutua* entre funciones y entre tipos, lo que obliga a separar la recolección de declaraciones de la verificación de los cuerpos.

1.2 Principios de diseño del compilador

El compilador descrito aquí está construido en torno a cinco principios de diseño explícitos, todos ellos consecuencia de un mismo compromiso de fondo: priorizar la transparencia de la maquinaria interna sobre la conveniencia inmediata. Las herramientas modernas de generación de *frontends* (LALRPOP, ANTLR, Bison) y de *backends* (Cranefield, GCC plugins) permiten obtener un compilador funcional en muy pocas líneas, pero a costa de ocultar precisamente los algoritmos que el documento se propone analizar.

D1. Transparencia algorítmica.

Cada fase implementa de forma explícita los algoritmos canónicos asociados a ella: construcción de Thompson en el *lexer*, cálculo de FIRST y FOLLOW por punto fijo y construcción canónica de LR(1) con fusión LALR(1) en el *parser*, propagación bidireccional de tipos con dos pasadas en el analizador semántico, *lowering* estructurado a SSA en el generador de código. Ninguna fase delega su núcleo algorítmico en una biblioteca externa.

D2. Compilación a código nativo, no interpretación.

El compilador es un traductor estricto: produce un binario ELF x86-64 enlazable, no un interpretador del AST ni una máquina virtual ad-hoc. La intención es validar la implementación contra la métrica más exigente posible —tiempos de ejecución comparables con el código C equivalente—, no contra una semántica abstracta.

D3. Seguridad de memoria del compilador mismo.

El compilador maneja estructuras de datos complejas con múltiples referencias cruzadas (tablas de símbolos jerárquicas, grafos de tipos, tablas LALR con cientos de estados). Se utilizó RUST como lenguaje anfitrión para eliminar de raíz una clase entera de errores que históricamente han plagado los compiladores escritos en C++: *dangling pointers*, *double frees*, lecturas de memoria inválida. El *wrapper inkwell* [10] aporta la misma garantía sobre el uso de la API C de LLVM.

D4. Reproducibilidad y *debuggability*.

Toda estructura de datos significativa —la gramática, la tabla LALR, el autómata, la jerarquía de tipos, el LLVM IR generado— es inspeccionable en tiempo de ejecución mediante volcados textuales legibles. La gramática se describe como una estructura de datos en memoria en lugar de un *DSL* externo, lo que permite imprimir cada producción con su identificador y depurar conflictos sin recompilar.

D5. Extensibilidad sintáctica y semántica.

El sistema de tipos, el AST y el generador de código fueron diseñados con anticipación a la introducción de extensiones. La extensión central documentada en este reporte —vectores con expresiones generadoras— añade tres formas sintácticas y un nuevo constructor de tipos paramétrico, y demuestra empíricamente que la organización modular es capaz de absorber esa adición sin reescrituras significativas.

2

Arquitectura del compilador

EL compilador adopta la organización en fases secuenciales que es habitual en la disciplina desde los primeros compiladores prácticos [18, 1]: el programa fuente atraviesa una sucesión de módulos, cada uno de los cuales lo transforma en una representación interna distinta hasta llegar al binario ejecutable. Este capítulo describe la arquitectura concreta del compilador implementado; los capítulos sucesivos entran en el detalle de cada fase.

2.1 El pipeline de compilación

La compilación es siempre una traducción *ahead of time*: el binario `hulk` recibe como argumento la ruta de un fichero `.hulk`, lo procesa por completo, escribe un fichero objeto en un directorio temporal y lo enlaza con la biblioteca de *runtime* mediante `gcc`, dejando en el directorio actual un ejecutable llamado `out.put`. El proceso es estrictamente secuencial: cada fase se ejecuta una sola vez, en orden, y sus errores se reportan según el código de salida fijado por el contrato de interfaz (1 para errores léxicos, 2 para sintácticos, 3 para semánticos).

La Figura 2.1 representa el pipeline tal como está implementado en `src/main.rs`. Las cajas verticales en línea continua corresponden a las fases que el compilador ejecuta en su orden de invocación; las cajas laterales en línea punteada representan las estructuras de datos que cada fase produce y la fase siguiente consume.

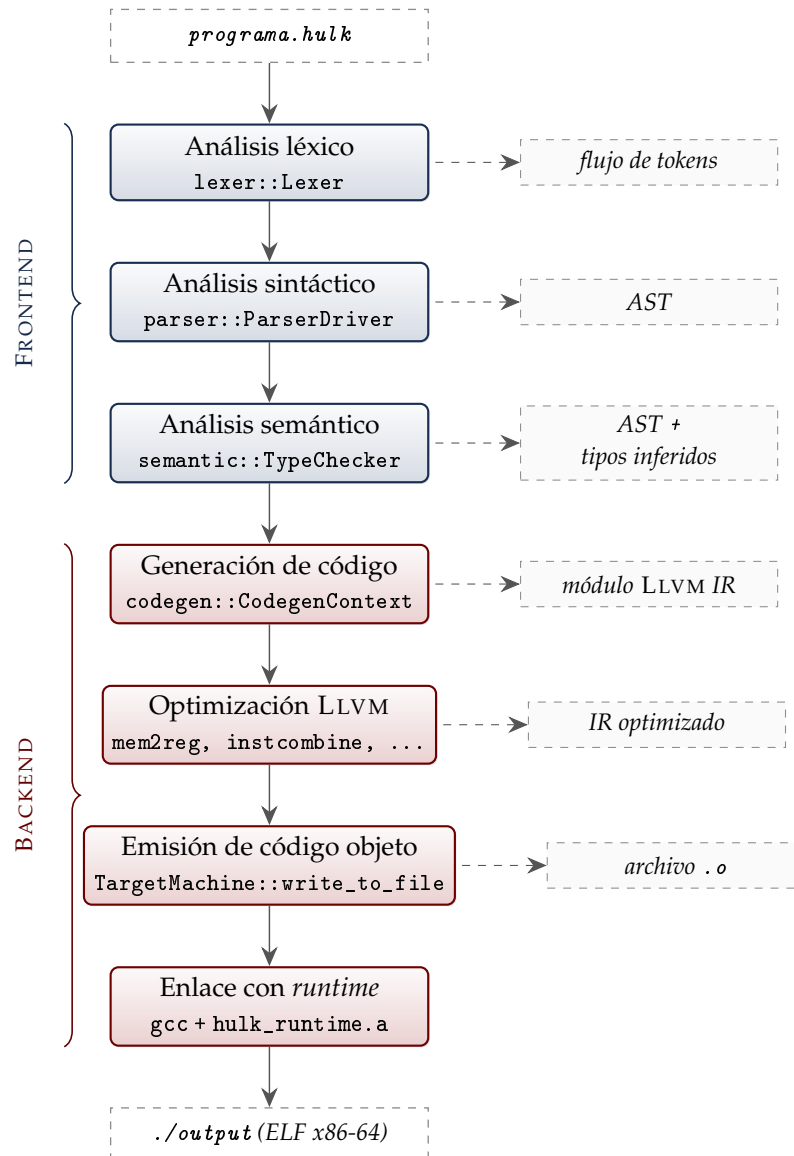


FIGURA 2.1. Pipeline de compilación implementado en `src/main.rs`. La columna central contiene las fases ejecutadas en orden; cada caja indica el módulo de Rust que realiza la transformación. Los recuadros laterales en línea punteada son las estructuras de datos que cada fase produce y la fase siguiente consume. Las llaves a la izquierda agrupan visualmente el *frontend* (azul) y el *backend* (rojo); la frontera entre ambos coincide con la producción del módulo de LLVM IR.

2.2 Las cuatro fases del *frontend*

El *frontend* comprende las tres primeras fases del compilador y se encarga de transformar el texto fuente en un AST anotado con tipos. La implementación reparte esta responsabilidad entre tres módulos de Rust con fronteras estrictas: `lexer`, `parser` y `semantic`. La Figura 2.1 muestra cada uno alimentando al siguiente sin retroalimentación.

Lexer. El módulo `lexer` reconoce tokens a partir del flujo de caracteres del fichero fuente. Su núcleo es un autómata finito no determinista construido por el algoritmo de Thompson a partir de las expresiones regulares declaradas para cada categoría léxica. Los autómatas de todos los patrones se combinan en un *MasterNFA*, del que el `lexer` toma decisiones por la

regla del máximo prefijo con desempate por prioridad. El resultado es un vector de tokens entregado por completo al *parser*; los tokens que encubren errores léxicos se marcan como `ERROR` sin interrumpir el reconocimiento, lo que permite reportar varios errores léxicos en una misma compilación.

Parser. El módulo `parser` consume la secuencia de tokens y construye el árbol sintáctico abstracto. Internamente se subdivide en cuatro submódulos: `grammar` define la gramática como una colección de producciones declarativas, `lalr` construye las tablas LALR(1) de `ACTION` y `GOTO` en tiempo de inicialización a partir de esa gramática, `engine` ejecuta el algoritmo *shift-reduce* sobre las tablas precalculadas, y `ast` define la jerarquía de nodos del árbol sintáctico abstracto. La construcción de las tablas —algoritmo canónico de DeRemer [3]— ocurre una sola vez al iniciarse el compilador.

Semantic. El módulo `semantic` verifica el AST y lo enriquece con información de tipos. Se compone de un submódulo de *sistema de tipos* —definiciones del enum `HulkType` y de la estructura `TypeHierarchy`—, una tabla de símbolos jerárquica con *scoping* estructurado, y un *type checker* que recorre el AST en dos pasadas: la primera registra las declaraciones de tipos, protocolos y funciones; la segunda visita los cuerpos, infiere los tipos ausentes y escribe los resultados en un mapa indexado por el identificador único de cada expresión. Al concluir, el análisis retorna una estructura con tres componentes: la jerarquía de tipos resuelta, las firmas de las funciones, y los tipos inferidos por expresión.

2.3 Las fases del backend

El *backend* —módulo `codegen`— traduce el AST tipado a un módulo de LLVM IR. La generación se orquesta en `codegen::jit`, que contiene los dos puntos de entrada del módulo: `execute_program_jit` para ejecución *just-in-time* usada en los tests, y `compile_to_binary` para la compilación AOT que produce el binario final. Ambas comparten el mismo flujo interno.

Lowering. La estructura `CodegenContext` encapsula el estado de la generación: el contexto, el módulo y el constructor de LLVM, una pila de tablas de símbolos con *scoping* léxico, el registro de *layouts* de structs y *vtables* por tipo, y los mapas de tipos heredados del analizador semántico. La operación `visit_program` construye el módulo LLVM en tres pasos sucesivos: declaración de las funciones externas del *runtime*, predeclaración de todas las funciones del programa (lo que habilita la recursión mutua sin reordenar el AST), y construcción de los *layouts* de tipos con asignación de *type tags* en orden DFS pre-orden. Sobre ese cimiento se compila el cuerpo de cada declaración (`lower_decl`) y, finalmente, la expresión de entrada del programa (`lower_expr`), envuelta en una función `__hulk_entry()` accesible desde la convención de llamada de C.

Optimización. El módulo LLVM producido se somete a un *pipeline* fijo de transformaciones aplicadas con el *new pass manager* de LLVM 17, declarado en `codegen::jit` como la cadena `"mem2reg, instcombine, reassociate, simplifycfg"`. `mem2reg` es el habilitador fundamental: promueve los patrones `alloca/store/load` emitidos por el *lowering* a registros virtuales SSA, lo que abre la puerta a las simplificaciones algebraicas y de control de

flujo aplicadas por los *passes* subsiguientes.

Emisión de código objeto. Tras la optimización, el módulo se escribe en un fichero objeto temporal mediante `TargetMachine::write_to_file`, configurado para la *triple nativa* de la máquina sobre la que se ejecuta el compilador (en la práctica, `x86_64-unknown-linux-gnu`).

Enlace. El compilador invoca finalmente `gcc` con tres argumentos: el fichero objeto recién emitido, el archivo `hulk_runtime.a` con las primitivas de *runtime*, y la bandera `-lm` para enlazar `libm`. El resultado es un binario ELF x86-64 ubicado en `./output` que el sistema operativo carga y ejecuta sin más dependencias dinámicas que `libc` y `libm`.

2.4 Elección del lenguaje anfitrión: RUST

Tres lenguajes constituyen alternativas razonables para implementar un compilador moderno con *backend* basado en LLVM: C, C++ y RUST. La elección recayó sobre RUST por cuatro razones técnicas concretas.

Seguridad de memoria sin coste de *garbage collection*. El compilador maneja estructuras de datos complejas con múltiples referencias cruzadas: tablas de símbolos jerárquicas, grafo de tipos con herencia, tabla de parseo LALR con cientos de estados. RUST garantiza la ausencia de *dangling pointers*, *double frees* y carreras de datos en tiempo de compilación, lo que elimina una clase entera de *bugs* históricamente difíciles de depurar en compiladores escritos en C++.

Sistema de tipos algebraico. Los enumerados con datos asociados (`enum`) y la verificación exhaustiva de patrones convierten la representación del AST en una tarea natural y segura. La ausencia de un caso en un `match` se reporta como error en tiempo de compilación, lo que evita bugs por casos faltantes.

Interfaz segura con LLVM. La biblioteca *inkwell* [10] encapsula la API C de LLVM en abstracciones de RUST que aprovechan el sistema de tipos para impedir operaciones inválidas (p. ej. construir una instrucción en un bloque básico ya terminado). Esto contrasta con el uso directo de `libLLVM` desde C++, donde tales errores producen *segfaults* difíciles de diagnosticar.

Ecosistema y herramientas. El gestor de paquetes cargo, la integración nativa con sistemas de pruebas y la disponibilidad de bibliotecas de calidad de producción permiten enfocar el esfuerzo en la lógica del compilador en lugar de en infraestructura.

3

Análisis léxico

EL primer paso de cualquier compilador consiste en transformar el flujo de caracteres del fichero fuente en una secuencia de *tokens* con significado. Esta fase se denomina *análisis léxico* o *tokenización*, y en este compilador se ha implementado mediante autómatas finitos no deterministas.

3.1 Arquitectura: lexer basado en NFA

3.1.1 Construcción de Thompson

La construcción clásica de Thompson [2] permite traducir una expresión regular cualquiera en un autómata finito no determinista con dos propiedades clave:

- exactamente un estado inicial y exactamente un estado de aceptación;
- las transiciones epsilon (ϵ) habilitan la composición modular de subautómatas sin alterar el alfabeto reconocido.

La construcción procede inductivamente sobre la estructura de la expresión regular, generando fragmentos NFA que se combinan según los operadores (concatenación, alternación, clausura). El tamaño del NFA resultante es lineal en el tamaño de la expresión regular original, lo que la hace adecuada para uso práctico.

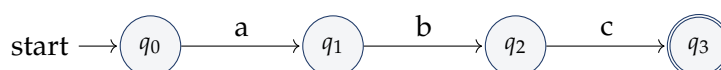


FIGURA 3.1. Fragmento NFA generado por Thompson para la expresión regular *abc*.

3.1.2 Ejecución sin determinización

El enfoque clásico convierte el NFA en un DFA antes de usarlo mediante la *construcción de subconjuntos*, lo que produce un autómata más grande pero más rápido de simular. En este compilador se ha optado por **ejecutar el NFA directamente**, calculando los subconjuntos de estados activos en cada paso. Esta decisión se justifica por:

- la simplicidad del código resultante;
- el tamaño modesto de los ficheros fuente HULK (decenas a miles de líneas);
- la ausencia de la fase de determinización, que podría duplicar exponencialmente el número de estados en el peor caso.

3.1.3 El MasterNFA y la regla del máximo prefijo

Para que el *lexer* reconozca todos los tipos de tokens a la vez, los NFA individuales de cada patrón se combinan en un **MasterNFA**: un autómata con un nuevo estado inicial y transiciones epsilon hacia los estados iniciales de cada NFA componente. La Figura 3.2 ilustra el esquema.

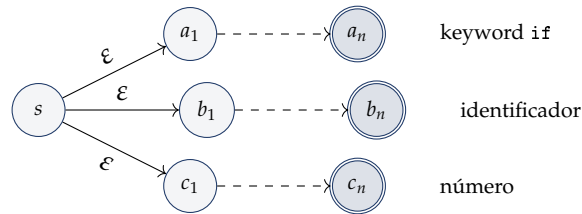


FIGURA 3.2. MasterNFA: estado inicial común con transiciones epsilon a los NFA individuales por patrón.

Cuando varios estados de aceptación son alcanzables simultáneamente, la regla del *máximo prefijo* (*maximal munch*) selecciona el patrón que reconoce la cadena más larga. En caso de empate (por ejemplo, *if* podría interpretarse como *keyword* o como identificador), la **prioridad** numérica asignada a cada patrón resuelve la ambigüedad: los *keywords* tienen mayor prioridad que los identificadores.

3.2 Conjunto de tokens reconocidos

HULK define 43 tipos de token agrupados en cinco categorías. La Cuadro 3.1 resume la clasificación.

TABLA 3.1. Conjunto de tokens del lenguaje HULK.

CATEGORIA	TOKENS
Palabras reservadas	let, in, if, elif, else, while, for, function, type, new, self, inherits, base, protocol, extends, is, as, true, false, null
Operadores aritméticos	+, -, *, /, %, ^, **
Operadores lógicos	&, , !
Operadores de comparación	==, !=, <, >, <=, >=
Operadores especiales	@, @@, :=, +=, -=, *=, /=, %=, =>, ->, ++, -
Delimitadores	(,), {, }, [,], ,, ;, :, .
Literales	números en notación decimal y flotante, cadenas entre comillas dobles, caracteres entre comillas simples, identificadores alfanuméricos

3.3 Secuencias de escape

Los literales de cadena se reconocen como una secuencia entre comillas dobles que admite cualquier carácter, salvo la propia comilla doble sin escapar. El *lexer* emite el lexema en bruto

—comillas incluidas— y delega en la fase de acciones semánticas del parser el retiro de las comillas envolventes y la decodificación de las secuencias de escape. Esta separación de responsabilidades preserva la posición original del lexema en el código fuente, lo que mantiene intactas las coordenadas usadas en los mensajes de error, y concentra la manipulación de cadenas en un único punto del pipeline.

El *lexer* valida que toda contrabarra dentro de un literal vaya seguida de uno de los caracteres reconocidos: una contrabarra seguida de un carácter ajeno al conjunto admitido produce un error léxico con la columna exacta del fallo. Las secuencias válidas y su traducción son:

```
\n -> salto de línea (U+000A)
\t -> tabulación horizontal (U+0009)
\" -> comilla doble
\\ -> contrabarra
```

LISTADO 3.1. Tabla de decodificación aplicada al construir `Literal::String`.

La decodificación se realiza al construir el nodo `Literal::String` en las acciones semánticas del parser: un bucle sobre los caracteres del lexema sustituye cada par `\c` por su carácter correspondiente y copia el resto sin cambios. Por tanto, un `print("\nhola")` imprime un salto de línea seguido de `hola`, como cabe esperar.

3.4 Recuperación de errores

Cuando el *lexer* encuentra un carácter que no inicia ningún patrón válido, emite un token especial `ERROR` en lugar de detener la compilación. Esto permite que el *parser* continúe avanzando y eventualmente reporte múltiples errores en una misma invocación. El formato del mensaje sigue el contrato oficial de la interfaz de evaluación:

```
! (línea,columna) LEXICAL: unexpected character 'X'
```

LISTADO 3.2. Formato estándar de error léxico.

4

Análisis sintáctico

UNA vez convertido el fuente en un flujo de tokens, la fase de análisis sintáctico verifica que dicho flujo se ajuste a la gramática del lenguaje y construye una representación estructurada del programa: el *árbol sintáctico abstracto* (AST).

4.1 Parser LALR(1) implementado desde cero

A pesar de la disponibilidad de generadores maduros como ANTLR, YACC, Bison o LALR-POP en el ecosistema RUST, el parser de este compilador está implementado manualmente sobre la base del principio de *transparencia algorítmica* (D1) enunciado en la introducción. Tres ventajas adicionales se derivan de esa decisión:

- **Inspeccionabilidad:** la gramática se representa como una estructura de datos RUST en lugar de procesarse desde un DSL textual externo. Esto permite imprimirla, recorrerla y analizar conflictos *shift/reduce* en tiempo de ejecución, sin pasos adicionales de compilación.
- **Cero dependencias externas:** el *build* del compilador no requiere invocar ni instalar generadores como YACC o Bison. La cadena de *build* se reduce a cargo *build*.
- **Control fino sobre la resolución de conflictos:** cuando la gramática introduce un conflicto deliberado —como el del separador `|` en las expresiones generadoras descritas en el Capítulo 8—, la resolución se programa directamente en el constructor de la tabla, sin necesidad de declarar precedencias en un metalenguaje externo.

4.2 Pipeline del parser

El módulo de análisis sintáctico se compone de cuatro capas claramente delimitadas que se describen a continuación.

4.2.1 Definición de la gramática

La gramática de HULK se especifica en dos formas equivalentes:

- un fichero `src/parser/grammar/hulk.grammar` en notación BNF extendida, legible para humanos y documentado con comentarios;
- una estructura `Vec<Production>` en `hulk_grammar.rs` construida programáticamente, que es la versión efectivamente usada por el generador de tablas.

4.2.2 Cálculo de FIRST y FOLLOW

Los conjuntos $\text{FIRST}(\alpha)$ y $\text{FOLLOW}(A)$ se calculan mediante el algoritmo clásico de punto fijo. Recordamos las definiciones:

DEFINICIÓN: CONJUNTOS FIRST Y FOLLOW

Sea $G = (N, T, P, S)$ una gramática libre de contexto, $A \in N$ un no-terminal y $\alpha \in (N \cup T)^*$ una forma sentencial.

$$\begin{aligned}\text{FIRST}(\alpha) &= \{a \in T \mid \alpha \Rightarrow^* a\beta\} \cup \{\varepsilon \mid \alpha \Rightarrow^* \varepsilon\} \\ \text{FOLLOW}(A) &= \{a \in T \cup \{\$ \} \mid S \Rightarrow^* \alpha A a \beta\}\end{aligned}$$

El algoritmo de punto fijo itera sobre las producciones actualizando los conjuntos hasta que en una iteración completa no cambia ningún conjunto.

4.2.3 Construcción de la tabla LALR

La construcción sigue el algoritmo estándar [1, 14] y se realiza en tres pasos. Primero se construye la colección canónica de *items* LR(1) mediante un BFS desde el *item* inicial $[\text{Start} \rightarrow \bullet \text{Program } \$, \$]$ tomando clausuras y aplicando $\text{GOTO}(I, X)$ para cada par estado-símbolo. Para la gramática completa de HULK este paso produce del orden de 500–600 estados LR(1). Segundo, se aplica el paso característico del algoritmo LALR(1) [3]: dos estados con idéntico *núcleo* (igual conjunto de pares producción-punto, ignorando *lookaheads*) se funden en uno solo, uniando sus conjuntos de *lookaheads*. La fusión reduce el espacio de estados a aproximadamente **263** estados, una reducción de cerca del 55 %. Tercero, se rellenan las tablas $\text{ACTION}[s, t]$ y $\text{GOTO}[s, N]$ recorriendo los *items* de cada estado fusionado y se registran los conflictos detectados para su inspección posterior.

TABLA 4.1. Estadísticas del autómata de parseo construido para HULK.

MÉTRICA	VALOR
Producciones declaradas	~ 90
Estados LR(1) antes de fusión	500–600
Estados LALR(1) tras fusión por núcleo	263
Entradas no triviales en ACTION	~1 500
Entradas no triviales en GOTO	~800
Conflictos <i>shift/reduce</i> sin resolver	0
Conflictos <i>shift/reduce</i> resueltos	1

El único conflicto de la gramática se sitúa sobre el terminal $|$ y nace de la interacción entre el operador OR lógico y el separador de las expresiones generadoras introducidas por la extensión (Capítulo 8). Cuando el motor de parseo se encuentra en un estado con la pila de la forma $[[\text{Expr}]]$ y el siguiente terminal es $|$, el *lookahead* estándar de un símbolo no basta: $[\text{Expr} \mid \dots]$ puede continuar tanto en $[\text{Expr} \mid \text{Expr}]$ —un vector explícito cuyo primer elemento es un OR lógico— como en $[\text{Expr} \mid \text{Id in Expr}]$ —un generador—. El constructor de la tabla resuelve este caso particular introduciendo un *lookahead* adicional de dos tokens: si tras el $|$ viene un IDENTIFIER seguido de *in*, se opta por la regla del generador; en cualquier

otro caso, por la regla de OR. La resolución se realiza en el momento de la construcción de la tabla, no en tiempo de parseo, por lo que el motor sigue siendo estrictamente LALR(1) en ejecución.

4.2.4 Motor de parseo

El motor implementa el algoritmo shift-reduce sobre las tablas precomputadas. Mantiene una pila de estados y una pila de valores semánticos en paralelo. En cada reducción se invoca una acción semántica que combina los valores semánticos del lado derecho de la producción en un nodo del AST.

4.3 Jerarquía de precedencias

La especificación oficial de HULK no establece la precedencia de operadores de forma explícita. La gramática implementada codifica la precedencia estándar matemática directamente en la jerarquía de no-terminales, evitando la necesidad de declaraciones de precedencia separadas. El Listado 4.1 muestra el esquema.

```
Expr      -> Assignment
Assignment -> OrExpr (':=' Assignment)?
OrExpr    -> AndExpr ('|' AndExpr)*
AndExpr   -> NotExpr ('&' NotExpr)*
NotExpr   -> '!' NotExpr | Equality
Equality  -> Comparison (('==' | '!=') Comparison)*
Comparison -> Concat (('<' | '>' | '<=' | '>=') Concat)*
Concat    -> Additive (('@' | '@@') Additive)*
Additive  -> Term (('+' | '-') Term)*
Term      -> Power (('*' | '/' | '%') Power)*
Power     -> Unary ('~' Power)?      (* asociatividad derecha *)
Unary     -> '-' Unary | Postfix
Postfix   -> Atom ('.' id '(' args ')' | '.' id | '[' Expr ']')*
Atom      -> literal | id | '(' Expr ')' | New | If | Let | ...
```

LISTADO 4.1. Codificación de precedencias en la gramática.

La asociatividad derecha de la potencia se obtiene mediante la recursión a la derecha en la regla Power: la expresión 2^3^2 se parsea como $2^{(3^2)} = 2^9 = 512$, no como $(2^3)^2 = 64$.

4.4 El árbol sintáctico abstracto

El AST está definido en `src/parser/ast/` mediante enumerados RUST con datos asociados. El nodo raíz es `Program`, que contiene una lista de declaraciones (`Decl`) y una expresión de entrada (`Expr`).

```
1 pub struct Program {
2     pub declarations: Vec<Decl>,
3     pub entry:      Expr,
4 }
5
6 pub struct Expr {
7     pub kind: ExprKind,
8     pub id:   u32,      // identificador único para la tabla de tipos
```

```

9      pub span: Span,      // línea y columna para mensajes de error
10  }
11
12  pub enum ExprKind {
13      Literal(Literal),
14      Identifier { name: String },
15      Binary(BinaryExpr),      Unary(UnaryExpr),
16      Postfix(PostfixExpr),    Assign(AssignExpr),
17      Let(LetExpr),           Block(BlockExpr),
18      If(IfExpr),             While(WhileExpr),
19      For(ForExpr),           Call(CallExpr),
20      MethodCall(MethodCallExpr),
21      Access(AccessExpr),      Index(IndexExpr),
22      New(NewExpr),            Base,
23      Is { expr: Box<Expr>, type_name: TypeName },
24      As { expr: Box<Expr>, type_name: TypeName },
25      Vector(VectorExpr),      // <-- extensión propia (cap. 8)
26  }

```

LISTADO 4.2. Estructura central del AST.

Cada Expr lleva un identificador único `id` que se utiliza como clave en la tabla `expr_types: HashMap<u32, HulkType>` que el analizador semántico construye. El generador de código consulta esta tabla para conocer el tipo inferido de cada subexpresión sin tener que reinferir.

5

Análisis semántico

CONSTRUIDO el AST, la siguiente fase verifica que el programa tenga sentido a nivel de tipos, alcances y referencias. Esta fase es responsable de detectar errores como uso de variables no declaradas, asignaciones entre tipos incompatibles, llamadas a funciones con número o tipo incorrecto de argumentos, y violaciones de la conformidad estructural a protocolos.

5.1 Estrategia en dos pasadas

El analizador semántico opera en dos pasadas claramente diferenciadas sobre la lista de declaraciones del programa.

Pasada 1 — Recolección de declaraciones. Todas las firmas de funciones, los miembros de tipos (atributos y métodos) y los cuerpos de protocolos se registran en la estructura `TypeHierarchy`. Los miembros sin anotación de tipo reciben un marcador `HulkType::Unknown` que será resuelto en la segunda pasada. Esta pasada es indispensable para soportar **recursión mutua** entre funciones y tipos: en el momento de verificar el cuerpo de `foo`, la firma de `bar` ya está disponible aunque `bar` esté declarada después.

Pasada 2 — Verificación de cuerpos y propagación de tipos. Cada cuerpo de función, cada inicializador de atributo y cada cuerpo de método se verifica. El tipo inferido del cuerpo se compara con la anotación declarada si existe; si no existe, el tipo inferido se **escribe de vuelta** en la `TypeHierarchy` reemplazando el marcador `Unknown`.

Esta escritura es *crítica* para la fase de generación de código. Si un atributo `val = start` no tiene anotación de tipo, la pasada 1 almacena `Unknown`; la pasada 2 infiere `Number` a partir del inicializador; la escritura de vuelta hace que `Number` esté disponible para el generador de código. Sin esta escritura, el código no podría determinar el tipo LLVM correcto del campo.

```
1 // Después de inferir el tipo del campo en la pasada 2:
2 if let Some(type_info) = self.types.types.get_mut(type_name) {
3     if let Some(attr) = type_info.attributes.get_mut(field_name) {
4         if *attr == HulkType::Unknown {
5             *attr = inferred_type.clone();
6         }
7     }
8 }
```

LISTADO 5.1. Propagación del tipo inferido en la pasada semántica.

5.2 Sistema de tipos

El sistema de tipos es **nominal** (la identidad de los tipos depende de su nombre y no de su estructura) con **subtype polymorphism** [6].

5.2.1 Categorías de tipo

- **Primitivos:** Number (IEEE-754 *double* de 64 bits), Boolean (un bit), String (puntero a cadena C *null*-terminada), Null (tipo singleton).
- **Definidos por el usuario:** introducidos mediante `type` con herencia simple opcional. Todos heredan transitivamente de `Object`.
- **Protocolos:** tipos de interfaz que listan firmas de métodos. La conformidad es **estructural**: un tipo conforma a un protocolo si declara todos sus métodos con firmas compatibles, sin necesidad de declarar la relación explícitamente.
- **Vectores:** el tipo `Vector<T>` para colecciones homogéneas de elementos de tipo `T` (vease Capítulo 8).
- **Tipos auxiliares:** `Unknown` (marcador en la pasada 1), `Never` (tipo de error para evitar cascadas de errores falsos).

5.2.2 Relación de conformidad

La relación de *conformidad* $\preceq$ entre tipos formaliza el subtipado. Se define inductivamente y opera tanto sobre la jerarquía nominal (tipos definidos por el usuario) como sobre la jerarquía estructural (protocolos).

DEFINICIÓN: CONFORMIDAD DE TIPOS

Sean A, B tipos del lenguaje HULK. Se dice que A *conforma a* B , notación $A \preceq B$, si y sólo si se cumple alguna de las siguientes condiciones:

$A = B$	(reflexividad)
A hereda transitivamente de B	(subtipado nominal)
B es protocolo y A lo implementa	(conformidad estructural)
$A = \text{Null}$ y B es nominal o <code>Object</code>	(nulabilidad)
$A = \text{Vec}\langle S \rangle$, $B = \text{Vec}\langle T \rangle$, $S \preceq T$	(covarianza de vectores)
$B = \text{Object}$	(raíz universal)

La conformidad estructural a un protocolo P no se reduce a una verificación puramente sintáctica de presencia de métodos: cada firma debe ser *compatible* con la del protocolo, donde compatibilidad significa **covarianza en el tipo de retorno y contravarianza en los parámetros**. Estas reglas son las clásicas para preservar la seguridad de la sustitución en presencia de subtipado, debidas originalmente a Liskov y Wing [6].

DEFINICIÓN: COMPATIBILIDAD DE FIRMAS CON VARIANZA

Sean $\sigma_A = (\bar{p}_A) \rightarrow r_A$ la firma del método declarado en el tipo candidato y $\sigma_P =$

$(\bar{p}_P) \rightarrow r_P$ la firma exigida por el protocolo. σ_A es compatible con σ_P si y sólo si:

$$\begin{array}{ll} |\bar{p}_A| = |\bar{p}_P| & \text{(aridad)} \\ r_A \preceq r_P & \text{(retorno covariante)} \\ \forall i. p_P^{(i)} \preceq p_A^{(i)} & \text{(parámetros contravariantes)} \end{array}$$

Estas reglas se implementan en el método `signatures_protocol_compatible` de la jerarquía de tipos: la conformidad se decide recorriendo los métodos del protocolo (y los del protocolo padre, si lo tiene) y delegando en la relación `conforms` para cada par de tipos. La búsqueda del método sube por la cadena de herencia del candidato, de modo que un método heredado satisface igual de bien la firma exigida que uno propio. El verificador atajará la búsqueda en cuanto detecte que el padre del candidato ya conforma al protocolo, evitando reexaminar firmas heredadas.

5.2.3 Unificación de tipos en condicionales: el ancestro común más cercano

Una situación recurrente en HULK es la determinación del tipo de una expresión `if-elif-else` cuyas ramas devuelven valores de tipos relacionados pero no idénticos. La regla aplicada es la habitual en lenguajes con subtipado: el tipo de la expresión es el *ancestro común más cercano* (*lowest common ancestor*, *lca*) de los tipos de todas las ramas.

DEFINICIÓN: ANCESTRO COMÚN MÁS CERCANO

$\text{lca}(A, B)$ se define como el tipo de menor profundidad en la jerarquía de herencia que satisface $A \preceq T \wedge B \preceq T$. Para n ramas se define recursivamente: $\text{lca}(T_1, T_2, \dots, T_n) = \text{lca}(\text{lca}(T_1, T_2), T_3, \dots, T_n)$.

El algoritmo implementado es directo: se recolecta el conjunto $\mathcal{A}(A)$ de todos los ancestros de A (incluido A) subiendo por la cadena de `TypeInfo.parent`, y se asciende desde B hasta encontrar el primer tipo que pertenece a $\mathcal{A}(A)$. La existencia de tal tipo está garantizada porque todos los tipos heredan transitivamente de `Object`, que actúa como elemento maximal del orden.

5.3 Funciones y constantes predefinidas

El compilador pre-registra el siguiente conjunto de elementos accesibles desde cualquier alcance, sin necesidad de importación:

TABLA 5.1. Funciones y constantes predefinidas del runtime HULK.

NOMBRE	DESCRIPCIÓN	TIPO
<code>print(x)</code>	Imprime el valor en stdout	<code>Object</code> $\rightarrow$ <code>Null</code>
<code>sqrt(x)</code>	Raíz cuadrada	<code>Number</code> $\rightarrow$ <code>Number</code>
<code>sin(x)</code>	Seno (radianes)	<code>Number</code> $\rightarrow$ <code>Number</code>
<code>cos(x)</code>	Coseno (radianes)	<code>Number</code> $\rightarrow$ <code>Number</code>
<code>exp(x)</code>	Exponencial e^x	<code>Number</code> $\rightarrow$ <code>Number</code>
<code>log(b, x)</code>	Logaritmo en base b	<code>(Number, Number)</code> $\rightarrow$ <code>Number</code>
<code>rand()</code>	Aleatorio uniforme en $[0, 1)$	<code>Number</code>
<code>range(s, e)</code>	Rango iterable $[s, e)$	<code>(Number, Number)</code> $\rightarrow$ <code>Range</code>
<code>PI, E</code>	Constantes matemáticas	<code>Number</code>
<code>true, false</code>	Literales booleanos	<code>Boolean</code>

5.4 Reporte de errores semánticos

Los errores semánticos se acumulan en un vector `Vec<SemanticError>` y se reportan todos juntos al final del análisis. Esta estrategia evita la frustración del programador que debe corregir un error a la vez. Cada `SemanticError` incluye el `Span` de la expresión o declaración causante, lo que produce mensajes con posición precisa:

```
! (12,5) SEMANTIC: type 'Cat' does not implement method 'speak' of
    protocol 'Animal'
! (18,3) SEMANTIC: cannot assign value of type 'String' to variable of
    type 'Number'
! (24,9) SEMANTIC: function 'foo' expects 2 arguments, 3 given
```

6

Generación de código

COMPLETADO el análisis semántico, el compilador dispone de un AST anotado con tipos y de una jerarquía de tipos verificada. La fase de generación de código traduce este AST en un módulo de LLVM Intermediate Representation, listo para ser optimizado y compilado a código máquina nativo.

6.1 LLVM IR mediante *inkwell*

LLVM [4] es la infraestructura de compilación más utilizada del mundo. Provee una representación intermedia tipada y de bajo nivel, un conjunto extensible de *passes* de optimización, y *backends* para una variedad de arquitecturas (x86-64, ARM, RISC-V, WebAssembly, etc.).

La biblioteca *inkwell* es un *wrapper* RUST de la API C de LLVM que aprovecha el sistema de tipos del lenguaje para impedir construcciones inválidas en tiempo de compilación. Por ejemplo, los *builder* de instrucciones rechazan operaciones sobre bloques básicos ya terminados, y los valores están parametrizados por el Context de LLVM para impedir mezclas entre contextos distintos.

El estado de la generación de código se centraliza en `CodegenContext`, una estructura que agrupa:

- `context`, `module`, `builder`: los tres objetos centrales de la API LLVM.
- `symbols`: pila de alcances para variables locales.
- `type_registry`: layouts de structs LLVM y vtables por tipo HULK.
- `type_hierarchy`: jerarquía de tipos del analizador semántico, accesible para resolución de métodos en herencia.
- `expr_types`: tipos anotados por el verificador, consultados durante la generación.
- `current_function`, `self_ptr`, `current_type_name`: contexto de la función o método en compilación.

6.2 Representación de valores: el enum `CgValue`

Dado que HULK tiene tipos heterogéneos (`Number` es escalar, `String` y los objetos son punteros), no existe un único tipo LLVM universal. La representación uniforme se obtiene mediante un enum RUST que transporta el valor LLVM junto con su categoría de tipo HULK:

```
1 pub enum CgValue<'ctx> {  
2     Number(FloatValue<'ctx>), // LLVM: double  
3     Bool(IntValue<'ctx>), // LLVM: i1
```



```

4   Str(PointerValue<'ctx>),      // LLVM: ptr a cadena C
5   Object(PointerValue<'ctx>),   // LLVM: ptr al struct del tipo
6   Vector(PointerValue<'ctx>),   // LLVM: ptr al buffer en heap
7   Null,                        // puntero nulo
8   Void,                        // sin valor (efectos de lado)
9 }

```

LISTADO 6.1. Representación de valores en codegen.

Las coerciones entre variantes se realizan en `coerce_arg`, que inserta las instrucciones LLVM necesarias: `Bool` a `Number` usa `uitofp`, cualquier tipo primitivo a `ptr` para paso a `print(x: Object)` se empaqueta mediante un `alloca + store`.

6.3 Representación de objetos en memoria

Todo tipo definido por el usuario se representa como un `struct` LLVM con una distribución de campos uniforme. Los dos primeros campos son metadatos comunes a todos los objetos; los siguientes son los campos declarados por el usuario en orden *padre-primero*.

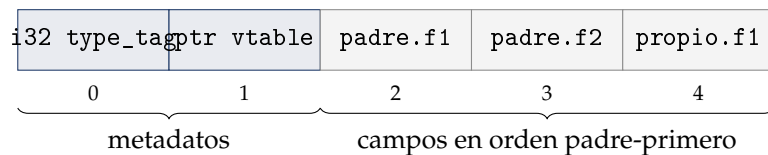


FIGURA 6.1. Layout en memoria de un objeto HULK que hereda de un padre con dos campos.

El orden *padre primero* es deliberado: garantiza que un puntero a un objeto del tipo `Dog` (que hereda de `Animal`) sea **bit-compatible** con un puntero a `Animal` respecto a los campos heredados. Esto evita el costo de reinterpretación de bits o de tablas de desplazamiento que serían necesarias con otros layouts.

6.4 Tablas de métodos virtuales (vtables)

El polimorfismo dinámico de HULK se implementa mediante *vtables*, la técnica estándar utilizada por C++, Java (compilado JIT) y otros lenguajes con herencia y polimorfismo.

Para cada tipo se crea una **vtable global** (un global LLVM) que contiene punteros a los métodos del tipo, en un orden fijo: primero los métodos heredados del padre, luego los nuevos. Cuando un tipo hace *override* de un método del padre, la posición en la vtable se conserva pero el puntero se reemplaza por el del nuevo método.

```

1 %VTable_Dog = type { ptr, ptr }
2
3 @vtable_Dog = global %VTable_Dog {
4     ptr @__hulk_method_Dog_speak,      ; override
5     ptr @__hulk_method_Animal_move    ; heredado del padre
6 }

```

LISTADO 6.2. Vtable global para un tipo `Dog` que redefine `speak` y hereda `move`.

El despacho de un método `x.speak()` sobre un valor `x` de tipo estático `Animal` se compila como un acceso indirecto en tres pasos: cargar el puntero a `vtable` desde el campo 1 del objeto, indexar al slot precalculado para el método `speak`, e invocar el puntero a función con la convención de llamada habitual (`self` en la primera posición, argumentos a continuación).

6.4.1 Despacho de métodos a través de un protocolo

El esquema de `vtables` es suficiente cuando el tipo estático del receptor es un tipo nominal: en ese caso, el orden de los `slots` es el mismo en todas las clases de la jerarquía y la posición del método se calcula en tiempo de compilación. Sin embargo, cuando el tipo estático es un protocolo `P`, los `slots` pueden diferir entre tipos conformantes: si `A` declara `{m,n}` y `B` declara `{n,m}`, la posición de `m` en cada `vtable` es distinta, y un acceso por índice fijo daría el método equivocado.

La estrategia elegida en este compilador resuelve esa indeterminación con un *switch* sobre el `type_tag` leído en tiempo de ejecución, generando una rama por cada tipo conforme conocido y, en cada rama, una llamada *directa* al método correspondiente. La construcción es local a la expresión de despacho —no requiere tablas globales adicionales— y se beneficia de la optimización clásica que LLVM aplica a los `switch`: tras `simplifycfg` se transforma en una *jump table*, lo que asintóticamente tiene el mismo coste que un acceso a `vtable`. El bloque `default` del `switch` se marca `unreachable` porque el verificador de tipos garantiza que el receptor realmente conforma al protocolo:

```

1  %tag = load i32, ptr %b
2
3  switch i32 %tag, label %proto_unreachable [
4      i32 3, label %proto_case_MBox
5      i32 5, label %proto_case_Cylinder
6  ]
7
8  proto_case_MBox:
9      %r1 = call f64 @__hulk_method_MBox_measure(ptr %b)
10     br label %proto_merge
11
12  proto_case_Cylinder:
13      %r2 = call f64 @__hulk_method_Cylinder_measure(ptr %b)
14      br label %proto_merge
15
16  proto_unreachable:
17      unreachable ; semánticamente inalcanzable
18
19  proto_merge:
20      %result = phi f64 [ %r1, %proto_case_MBox ], [ %r2, %proto_case_Cylinder
    ]

```

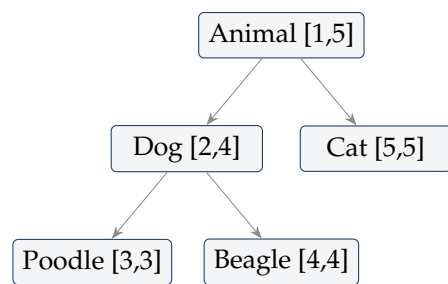
LISTADO 6.3. Despacho a través de un protocolo `Measurable` con dos tipos conformes.

La decisión entre despacho por `vtable` (para tipos nominales) y despacho por *switch* (para protocolos) se toma en `method_dispatch` del módulo `codegen::context` consultando la categoría sintáctica del tipo estático del receptor.

6.5 Etiquetas de tipo en orden DFS

El operador `is` (`x is Dog`) es una operación frecuente en código OOP con herencia, y se utiliza también internamente en la implementación del operador `as` (*downcast*). Implementarlo eficientemente requiere un invariante no trivial sobre la asignación de etiquetas de tipo.

Asignación DFS pre-orden. Durante la compilación, los tipos reciben **etiquetas enteras en orden de recorrido en profundidad de la jerarquía de herencia**, en pre-orden. El tipo padre recibe su etiqueta antes que sus hijos. Como consecuencia, **todos los subtipos directos e indirectos de un tipo T forman un intervalo contiguo** $[\min_tag_T, \max_tag_T]$ en el espacio de etiquetas.



poodle is Dog: $2 \leq 3 \leq 4$ ✓
 poodle is Animal: $1 \leq 3 \leq 5$ ✓
 cat is Dog: $2 \leq 5 \leq 4$ ✗

FIGURA 6.2. Asignación DFS pre-orden de etiquetas de tipo. El test `is` se reduce a una verificación de pertenencia a un intervalo, computable en tiempo constante.

Implementación del operador `is`. Gracias a esta asignación, el código LLVM generado para `x is Dog` es simplemente:

```

%tag = load i32, ptr %x
%ge  = icmp uge i32 %tag, 2      ; min_tag(Dog)
%le  = icmp ule i32 %tag, 4      ; max_tag(Dog)
%res = and i1 %ge, %le
  
```

Esto es $O(1)$ en tiempo de ejecución, sin consultar ninguna tabla ni recorrer la jerarquía. La técnica esta documentada en implementaciones eficientes de RTTI en C++ [5].

6.6 El operador `as`: downcast con verificación

El operador `x as T` realiza un *downcast*: convierte un valor de tipo base (por ejemplo `Animal`) al tipo derivado `T` (por ejemplo `Dog`). En HULK, esto debe ser *seguro*: si el objeto realmente no es de tipo `T`, la ejecución debe abortar con un mensaje de error en lugar de continuar con un puntero malformado.

El código generado primero verifica el rango de etiquetas (igual que en `is`). Si la verificación falla, llama a la función de *runtime* `hulk_type_error`, que imprime el mensaje de error y llama a `abort(3)`. Si la verificación tiene éxito, el puntero original se utiliza directamente: no se requiere reinterpretación de bits porque el layout es compatible por la organización padre-primerio.

6.7 La palabra clave `base()`

Dentro de un método, `base(args)` invoca la implementación del padre. A diferencia del despacho virtual — que cargaría la `vtable` y podría producir una llamada recursiva si el padre llama de vuelta al método del hijo —, `base()` es una **llamada directa estática**: el nombre de la función del padre se conoce en tiempo de compilación.

El código generador busca recursivamente en la jerarquía hasta encontrar el tipo que realmente implementa el método (no necesariamente el padre inmediato, si varios niveles intermedios solo lo heredan), y emite una llamada directa por nombre.

6.8 Pipeline de optimización

El IR generado por el codegen es deliberadamente sencillo y abundante en *allocas*: cada variable local se modela como una secuencia `alloca + store + load`. Esta forma facilita la generación correcta pero es ineficiente sin optimización.

Antes de la emisión de código objeto se aplica el *pipeline* del nuevo *pass manager* de LLVM 17 con la siguiente secuencia:

TABLA 6.1. Passes de optimización aplicados al IR.

PASS	EFEECTO
<code>mem2reg</code>	Promueve patrones <code>alloca/store/load</code> a registros SSA virtuales. Es el habilitador fundamental de todas las optimizaciones subsiguientes.
<code>instcombine</code>	Plegado de constantes, simplificación algebraica, eliminación de operaciones redundantes (por ejemplo, $x \oplus 0 = x$, $x \cdot 1 = x$).
<code>reassociate</code>	Reordena expresiones asociativas para maximizar la oportunidad de plegado por <code>instcombine</code> .
<code>simplifycfg</code>	Elimina bloques básicos muertos, fusiona bloques redundantes, simplifica el grafo de flujo de control.

Esta combinación produce un IR significativamente más compacto y próximo al código generado por compiladores maduros como `gcc -O2`.

7

Runtime y compilación final

EL código LLVM producido por el *frontend* no es autosuficiente: depende de un conjunto de funciones de soporte para operaciones como impresión, formato de números, asignación de cadenas y manejo de colecciones. Estas funciones forman la *biblioteca de runtime*, escrita en C y enlazada con cada ejecutable producido.

7.1 La biblioteca de runtime en C

El *runtime* se compila a un archivo estático `hulk_runtime.a` desde `runtime/hulk_runtime.c`. Provee las siguientes funciones, todas con enlace C y nombres no decorados para que sean accesibles desde el código generado.

TABLA 7.1. Funciones de la biblioteca de runtime.

FUNCIÓN	DESCRIPCIÓN
<code>hulk_print(ptr)</code>	Imprime cadena + <i>newline</i> a stdout.
<code>hulk_str_from_number(d)</code>	Convierte double a C string; formatea enteros sin parte decimal.
<code>hulk_str_concat(a,b)</code>	Concatenación para el operador @.
<code>hulk_str_concat_space(a,b)</code>	Concatenación con espacio para @@.
<code>hulk_str_eq(a,b)</code>	Igualdad de cadenas por valor.
<code>hulk_str_size(s)</code>	Longitud en bytes de la cadena.
<code>hulk_vec_alloc(n,sz)</code>	Alloca buffer: header (8 bytes) + $n \times 8$ bytes de elementos.
<code>hulk_vec_get(p,i,sz)</code>	Devuelve puntero al elemento <i>i</i> con verificación de límites.
<code>hulk_vec_size(p)</code>	Lee el header y devuelve el conteo.
<code>hulk_range_alloc(s,e)</code>	Crea iterador para el rango $[s,e)$.
<code>hulk_range_next(p)</code>	Avanza el iterador; devuelve true si hay más elementos.
<code>hulk_range_current(p)</code>	Valor actual del iterador.
<code>hulk_rand()</code>	Número aleatorio uniforme en $[0,1)$.
<code>hulk_type_error(p)</code>	Imprime el mensaje y aborta (as fallido).

7.2 Layout en memoria de vectores y rangos

Las dos estructuras compuestas del *runtime*, **vectores** y **rangos**, tienen layouts compactos diseñados para minimizar el número de *allocs* y maximizar la localidad de acceso.

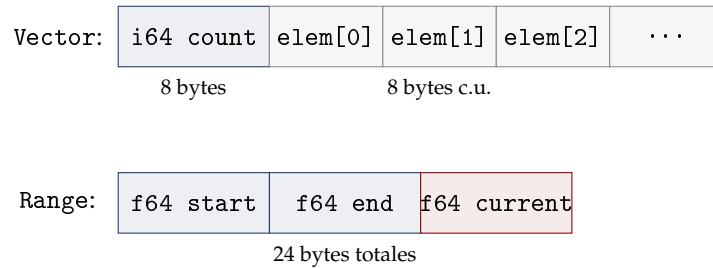


FIGURA 7.1. Layouts en memoria de vectores y rangos.

El layout de vector tiene exactamente **una palabra de cabecera** (8 bytes con el conteo) seguida del area de datos. Esto permite calcular el puntero al elemento i con una única operación de aritmética de punteros: $\text{vec} + 8 + 8*i$.

7.3 Pipeline AOT completo

Al invocar el compilador con `./hulk source.hulk`, el proceso completo es:

1. Análisis léxico, sintáctico y semántico del fuente.
2. Generación de LLVM IR para todas las declaraciones y la expresión principal.
3. Adición de una función `main` compatible con C que llama a `__hulk_entry()` y retorna 0.
4. Optimización del IR con el pipeline descrito en la Cuadro 6.1.
5. Emisión de código objeto a un fichero temporal mediante el método `write_to_file` de la `TargetMachine` de LLVM.
6. Enlace con gcc:

```
gcc /tmp/hulk_program.o hulk_runtime.a -o ./output -lm
```

El ejecutable resultante es un binario **ELF x86-64 autocontenido** que no requiere a LLVM en tiempo de ejecución. Sus únicas dependencias son la `libc` del sistema y la `libm`.

8

Extensiones del lenguaje implementadas

SOBRE el núcleo del lenguaje —expresiones, funciones, variables, control de flujo, tipos, herencia y verificación de tipos— nuestro compilador implementa tres extensiones que amplían de forma significativa las construcciones disponibles al programador: los **protocolos** con conformidad estructural, los **iterables** con el tipo predefinido `Range`, y los **vectores** con su sistema de literales, expresiones generadoras y construcciones de asignación dimensionada. Este capítulo se dedica a describir cada una de ellas.

Las tres extensiones están profundamente entrelazadas: los vectores implementan el protocolo iterable, los iterables se exponen al programador mediante el operador de anotación `T*` cuya semántica se apoya en el protocolo, y la conformidad estructural de los tipos es la maquinaria que hace todo eso uniforme. Tratarlas juntas en un único capítulo permite, por tanto, exponer su diseño como un sistema coherente y no como una colección de incorporaciones inconexas. Para cada una se describe lo que hace, cómo se implementa, qué soluciones equivalentes ofrecen otros lenguajes contemporáneos y qué decisiones de diseño distinguen nuestra implementación de las alternativas que se descartaron.

8.1 Protocolos: tipado estructural en tiempo de compilación

8.1.1 Motivación

Un protocolo en HULK es una colección de firmas de método sin implementación. Un tipo concreto satisface el protocolo si declara todos sus métodos con firmas compatibles, sin necesidad de declarar explícitamente que lo implementa. Esta noción —el tipado *estructural* en contraposición al *nominal*— es la pieza que permite escribir funciones genéricas sobre familias heterogéneas de tipos sin recurrir a la jerarquía nominal por herencia: una función que recibe un parámetro de tipo `Drawable` puede operar sobre cualquier objeto que sepa dibujarse, con independencia de su tipo concreto.

La motivación de incluir esta extensión en el compilador es doble. Primero, los protocolos son el único mecanismo del lenguaje que rompe la rigidez de la jerarquía nominal: dos tipos sin relación de herencia pueden conformar con el mismo protocolo si casualmente comparten una firma de método. Segundo, otras extensiones posteriores —en particular los iterables y los funtores— se definen *en términos de* protocolos predefinidos, de modo que sin protocolos esas extensiones no serían representables. La implementación de los protocolos es, por tanto, condición previa de las demás.

8.1.2 Sintaxis, AST y sinónimo léxico *interface*

La gramática de los protocolos sigue una forma simple y previsible:

```

ProtocolDecl -> 'protocol' IDENTIFIER '{' ProtocolBody '}'
              | 'protocol' IDENTIFIER 'extends' TypeName '{' ProtocolBody
              '}',

```

LISTADO 8.1. Producciones gramaticales para la declaración de protocolos.

El cuerpo de un protocolo es una lista de firmas de método: nombres con sus listas de parámetros tipados y un *tipo de retorno obligatorio*, a diferencia de las declaraciones de método de un tipo concreto, donde el retorno puede omitirse y el verificador lo infiere a partir del cuerpo. Esta obligatoriedad es estructural y no incidental: como un protocolo no posee implementación, no hay información a partir de la cual inferir.

El parser construye, para cada declaración, un nodo del AST con tres campos significativos: el nombre del protocolo, el nombre del protocolo padre (si lo hay) y la lista de firmas de método:

```

1 pub struct ProtocolDecl {
2     pub name:      String,
3     pub extends:   Option<TypeName>,
4     pub methods:   Vec<MethodSignature>,
5     pub span:      Span,
6 }
7
8 pub struct MethodSignature {
9     pub name:      String,
10    pub params:     Vec<Param>,
11    pub return_type: TypeName,
12    pub span:       Span,
13 }

```

LISTADO 8.2. Representación AST de una declaración de protocolo.

Sinónimo léxico *interface*. La palabra clave canónica del lenguaje para introducir un protocolo es `protocol`. Sin embargo, el concepto que el lenguaje denomina *protocolo* se conoce universalmente como *interface* en los lenguajes orientados a objetos contemporáneos: Java, C#, TypeScript, Kotlin y Swift utilizan `interface` o `protocol` de manera intercambiable en su literatura introductoria. Para reducir la fricción cognitiva de un programador con experiencia en cualquiera de estos lenguajes, nuestra implementación admite `interface` como *alias léxico* estricto de `protocol`.

La modificación se concentra exclusivamente en el analizador léxico, donde la expresión regular asociada al terminal de la palabra clave de protocolo admite ambas alternativas:

```

1 TokenDefinition {
2     token_type: TokenType::KW_PROTOCOL,
3     regex: "protocol|interface",
4     skippable: false,
5 },

```

LISTADO 8.3. Alias léxico para el terminal de protocolo.

El autómata maestro construido por la construcción clásica de Thompson [2] incorpora ambas alternativas como caminos paralelos desde el estado inicial del subautómata del

terminal; el efecto tras la simulación es que el flujo de tokens contiene en ambos casos el mismo terminal. La gramática, las estructuras del AST, el verificador de tipos y el generador de código no se modifican: las dos formas son indistinguibles a partir del flujo de tokens. Una declaración `interface T { ... }` produce exactamente el mismo nodo que `protocol T { ... }` y conforma al mismo tipo en el sistema de tipos.

8.1.3 Sistema de tipos y conformidad estructural

En el sistema de tipos del compilador, los protocolos son un constructor de tipo de primer nivel junto con los demás tipos del lenguaje:

```
1 pub enum HulkType {
2     Number, StringT, Boolean, Null, Object,
3     Vector(Box<HulkType>),
4     UserDefined(String),
5     Protocol(String),
6     Unknown, Never,
7 }
```

Para cada declaración de protocolo que aparece en el programa, el verificador registra una entrada en una tabla de protocolos que contiene el nombre, el nombre del protocolo extendido (si lo hay) y el conjunto de firmas de método:

```
1 pub struct ProtocolInfo {
2     pub name: String,
3     pub extends: Option<String>,
4     pub methods: HashMap<String, FuncSignature>,
5 }
```

La pieza central del verificador es la función que decide si un tipo concreto `T` conforma con un protocolo `P`. Su lógica sigue tres reglas:

1. Si el padre nominal de `T` ya conforma con `P`, entonces `T` hereda la conformidad: la relación es congruente con la herencia declarada.
2. Para cada método declarado en `P`, debe existir un método homónimo en `T` con una *firma compatible* bajo las reglas de varianza.
3. Si `P` extiende un protocolo `Q`, entonces `T` debe conformar adicionalmente con `Q`.

Las reglas de varianza son las habituales para sistemas con subtipado de funciones: los parámetros son *contravariantes* y el retorno es *covariante*. La función auxiliar que implementa esta verificación recorre las firmas posición a posición y aplica la relación de conformidad en cada caso:

```
1 fn signatures_protocol_compatible(
2     &self,
3     type_sig: &FuncSignature,
4     proto_sig: &FuncSignature,
5 ) -> bool {
6     // Retorno COVARIANTE: type.return debe conformar a proto.return
7     if !self.conforms(&type_sig.return_type, &proto_sig.return_type) {
8         return false;
9     }
10    // Parametros CONTRAVARIANTES: proto.param debe conformar a type.param
11    for (_, type_param), (_, proto_param)
```

```

12         in type_sig.params.iter().zip(proto_sig.params.iter()) {
13         if !self.conforms(proto_param, type_param) {
14             return false;
15         }
16     }
17     true
18 }

```

LISTADO 8.4. Verificación de compatibilidad de firmas con varianza.

La intuición de la varianza es la siguiente. Si un protocolo P declara un método $m(x: \text{Animal}): \text{Mamifero}$, una implementación que firme $m(x: \text{Object}): \text{Perro}$ satisface el contrato: acepta más entradas (Object es más general que Animal , contravarianza) y produce salidas más específicas (Perro conforma con Mamifero , covarianza). Cualquier código cliente que pase un Animal a m estará pasando, en particular, un Object ; cualquier código cliente que asuma recibir un Mamifero aceptará sin problema un Perro .

8.1.4 Despacho dinámico en LLVM

Cuando una expresión del programa invoca un método sobre un valor cuyo tipo estático es un protocolo, el compilador no puede determinar en tiempo de compilación qué implementación concreta se ejecutará. La elección depende del *tipo dinámico* del receptor, conocido sólo en ejecución. El generador de código resuelve este caso mediante un despacho indirecto basado en una *etiqueta de tipo* almacenada en el primer campo de toda instancia de objeto.

La estructura emitida es la siguiente. Sea p el puntero al receptor, P el protocolo declarado y $\{T_1, T_2, \dots, T_k\}$ el conjunto de tipos que conforman con P en el módulo actual. El compilador carga el campo de etiqueta de tipo de p , emite una instrucción `switch` con un destino por cada T_i , y en cada destino genera una llamada directa a la función estática que materializa la implementación concreta del método en T_i . Los valores de retorno se reúnen en un nodo `phi` al final:

```

1  %proto_tag = load i32, ptr %p
2  switch i32 %proto_tag, label %proto_unreachable [
3      i32 1, label %proto_case_Square
4      i32 2, label %proto_case_Triangle
5      i32 3, label %proto_case_Circle
6  ]
7
8  proto_case_Square:
9      %r0 = call double @__hulk_method_Square_area(ptr %p)
10     br label %proto_merge
11
12  proto_case_Triangle:
13     %r1 = call double @__hulk_method_Triangle_area(ptr %p)
14     br label %proto_merge
15
16  proto_case_Circle:
17     %r2 = call double @__hulk_method_Circle_area(ptr %p)
18     br label %proto_merge
19
20  proto_unreachable:
21     unreachable
22

```

```

23 proto_merge:
24     %proto_ret = phi double [ %r0, %proto_case_Square ],
25                               [ %r1, %proto_case_Triangle ],
26                               [ %r2, %proto_case_Circle ]

```

LISTADO 8.5. Patrón LLVM IR para despacho de método de protocolo.

El bloque `proto_unreachable` no se ejecuta nunca porque el verificador de tipos garantiza, en tiempo de compilación, que todo objeto cuyo tipo estático sea un protocolo P tiene una etiqueta de tipo correspondiente a algún conformante de P . La instrucción `unreachable` de LLVM comunica esta garantía al optimizador y permite eliminar la rama `default` del `switch` sin pagar el coste de un test de seguridad.

La decisión de no usar *fat pointers*. Una alternativa clásica al despacho por etiqueta de tipo es la representación de los valores de tipo protocolo como *fat pointers*: un par formado por el puntero al objeto y un puntero a su tabla de métodos para ese protocolo. Esta es la convención de Rust con `dyn Trait` y de Go con sus `interface{}`. Nuestra implementación rechaza esta alternativa por una razón concreta: la `vtable` es una propiedad del *tipo*, no del par (*tipo*, *protocolo*). Cada tipo concreto tiene una única `vtable` con todos sus métodos virtuales; el despacho por protocolo se reduce a un `switch` que selecciona qué tipo concreto invoca su método estático, sin necesidad de duplicar tablas por (tipo, protocolo). El coste asintótico es comparable —tras la optimización a *jump table* de LLVM, $O(1)$ en ambos casos— pero el coste en bytes del módulo emitido es menor: no se almacenan tablas de métodos específicas por protocolo, sólo el conjunto de implementaciones estáticas por tipo.

8.1.5 Análisis comparativo

La construcción “conjunto de métodos abstractos al que un tipo puede conformar” tiene encarnaciones en virtualmente todos los lenguajes orientados a objetos contemporáneos, divididas según dos ejes ortogonales: *nominal vs. estructural* (¿se declara explícitamente la conformidad o se deriva de la presencia de los métodos?) y *estática vs. dinámica* (¿se verifica en compilación o en ejecución?).

TABLA 8.1. Comparativa del mecanismo de conformidad por lenguaje.

LENGUAJE	CONSTRUCCIÓN	CONFORMIDAD	VARIANZA
HULK	protocol/interface	Estructural, estática	Cov. retorno, contrav. arg.
Java	interface	Nominal, estática	Cov. retorno (desde Java 5)
C#	interface	Nominal, estática	Cov. retorno (desde C# 9)
Rust	trait	Nominal, estática	Sin varianza implícita
Swift	protocol	Nominal, estática	Cov./contrav. explícita
Go	interface	Estructural, estática	Cov. retorno, contrav. arg.
Scala	trait	Nominal, estática	Cov./contrav. explícita (+T/-T)
TypeScript	interface/type	Estructural, estática	Bivariante por defecto

La elección de HULK —estructural y estática— coincide con la de Go y TypeScript. La diferencia más visible con Go reside en la sintaxis: en Go la inferencia del protocolo es completamente implícita y no existe un mecanismo análogo a `extends` para protocolos. La diferencia con TypeScript reside en el modelo de varianza: TypeScript es bivariante por defecto en posiciones de parámetro de método (una decisión polémica documentada como un *footgun* en su manual), mientras que HULK sigue la convención matemáticamente correcta de Liskov [19]: covarianza en retornos, contravarianza en argumentos.

La diferencia con Java, C#, Rust, Swift y Scala es estructural vs. nominal. En esos lenguajes, un tipo debe declarar explícitamente *qué* interfaces o *traits* implementa. Esta declaración tiene ventajas —intención explícita, mejor diagnóstico de error— pero impone un coste en flexibilidad: dos jerarquías independientes que casualmente compartan una firma de método no pueden interoperar sin recurrir a wrappers explícitos (adaptadores). La conformidad estructural elimina esta fricción a cambio de una propiedad: la conformidad es *ambiental*, depende sólo de las firmas, y añadir o quitar un método en un tipo puede romper la conformidad de manera silenciosa.

8.1.6 Decisiones de diseño

Por qué *vtable* por tipo y no por protocolo. Cada tipo concreto en nuestra implementación tiene una única *vtable* con todos sus métodos virtuales. Una alternativa habitual —usada por C++ con herencia múltiple— consiste en mantener una *vtable* por cada combinación de (tipo, base virtual). La razón por la que HULK no necesita esta complejidad es que el lenguaje sólo admite *herencia simple*: cada tipo tiene a lo sumo un padre, y el conjunto de métodos virtuales heredados está totalmente ordenado por la cadena de herencia. El modelo de protocolos por `switch` sobre tipo concreto se compone cómodamente con *vtables* por tipo: el `switch` selecciona el tipo, la *vtable* de ese tipo selecciona el método.

Por qué calcular los conformantes en tiempo de compilación. La emisión del `switch` requiere conocer en compilación la lista completa de tipos que conforman con cada protocolo.

Esta enumeración es posible porque HULK no admite carga dinámica de tipos: todo el código del programa es conocido al momento de compilar. Lenguajes con carga dinámica de tipos como Java o C# no pueden recurrir a esta técnica y deben pagar el coste de mantener tablas indirectas (*itables* en la jerga de la JVM) que se consultan en ejecución. Para HULK, la enumeración estática es a la vez más rápida en ejecución y más simple de implementar.

8.2 Iterables y rangos: el protocolo *Iterable*

8.2.1 Motivación

El protocolo *Iterable* y el tipo predefinido *Range* forman la infraestructura del lenguaje para la iteración sobre colecciones. El protocolo declara dos métodos: *next()*: *Boolean*, que avanza el iterador interno y devuelve si quedan elementos, y *current()*: *T*, que devuelve el elemento actual. El ciclo *for* (*x in iter*) *body* es azúcar sintáctico que se baja al *while* equivalente que invoca *next* y *current* alternadamente. El operador *T** en una anotación de tipo permite que un parámetro acepte cualquier tipo que implemente el protocolo, sin atarse a una implementación concreta.

La motivación práctica es la unificación: una vez que *Range*, los vectores y cualquier tipo definido por el usuario implementan el protocolo, el ciclo *for* y las expresiones generadoras de vectores (Apartado 8.3) operan uniformemente sobre todos ellos.

8.2.2 El protocolo *Iterable* y el tipo *Range*

El protocolo *Iterable* se registra durante la inicialización del sistema de tipos, antes de procesar las declaraciones del programa fuente, como uno más de los tipos predefinidos del lenguaje:

```

1  self.protocols.insert("Iterable".into(), ProtocolInfo {
2      name:      "Iterable".into(),
3      extends:  None,
4      methods: {
5          let mut m = HashMap::new();
6          m.insert("next".into(), FuncSignature {
7              params: vec![], return_type: HulkType::Boolean,
8          });
9          m.insert("current".into(), FuncSignature {
10             params: vec![], return_type: HulkType::Object,
11         });
12         m
13     },
14 });

```

LISTADO 8.6. Registro del protocolo *Iterable* como tipo predefinido.

El tipo *Range* se registra a continuación como tipo predefinido, con implementaciones concretas de los dos métodos que refinan los tipos de retorno del protocolo gracias a la regla de covarianza ya discutida: la firma *Range::current(): Number* es admisible como implementación de *Iterable::current(): Object* porque *Number* conforma con *Object*.

El *runtime* de *Range* consta de tres funciones expuestas con ABI C. La representación en memoria es un bloque de 24 bytes con tres *f64* consecutivos: *start*, *end* y *current*.

```

1 // Layout: [f64 start][f64 end][f64 current] (24 bytes, alineado a 8)
2 // current arranca en start-1 para que el primer next() lo lleve a start.
3 pub extern "C" fn hulk_range_alloc(start: f64, end: f64) -> *mut u8 {
4     let layout = Layout::from_size_align(24, 8).unwrap();
5     let ptr = unsafe { alloc(layout) } as *mut f64;
6     unsafe { *ptr = start; *ptr.add(1) = end; *ptr.add(2) = start - 1.0; }
7     ptr as *mut u8
8 }
9
10 // Avanza current y devuelve si current < end.
11 pub extern "C" fn hulk_range_next(range: *mut u8) -> bool {
12     let ptr = range as *mut f64;
13     unsafe { *ptr.add(2) += 1.0; *ptr.add(2) < *ptr.add(1) }
14 }
15
16 // Devuelve el valor actual del iterador.
17 pub extern "C" fn hulk_range_current(range: *mut u8) -> f64 {
18     unsafe { *(range as *const f64).add(2) }
19 }

```

LISTADO 8.7. Runtime del tipo Range.

La inicialización de `current` a `start - 1` es un truco clásico de iteradores que permite implementar la regla *“next avanza antes de devolver”* sin un caso especial para la primera invocación: la primera llamada a `next` lleva `current` de `start - 1` a `start` y devuelve `true` si el rango es no vacío. La invariante mantenida tras toda invocación de `next` es que `current` apunta al elemento que la próxima invocación de `current` devolverá. Si `next` retorna `false`, el contenido de `current` es indefinido pero el cliente no debe leerlo (es invariante del protocolo, no del *runtime*).

8.2.3 La anotación *T** y la transpilación del *for-loop*

La anotación `Number*` —y, en general, `T*` para cualquier `T`— se reconoce en la gramática como una producción adicional del no-terminal `TypeName`:

```
TypeName -> Identifier '*'
```

El parser construye un nodo del AST con la variante específica para iterables, y el resolutor de tipos del verificador la traduce a `HulkType::Protocol("Iterable")`, descartando deliberadamente la información del tipo de elemento:

```
1 TypeName::Iterable { .. } => HulkType::Protocol("Iterable".into()),
```

Esta pérdida de información es deliberada: un parámetro de tipo `Number*` se compromete a aceptar cualquier iterable, sin discriminar por tipo de elemento. La consecuencia es que dentro del cuerpo de la función no es posible asumir que el elemento es `Number`: el verificador trata todos los elementos como `Object` y exige conversiones explícitas (`as`) cuando se requiere refinamiento. Si el programador quiere preservar la información del elemento, debe usar la anotación `Number[]` (vector de `Number`), discutida en Apartado 8.3.

Transpilación del `for`. El ciclo `for (x in iter) body` es azúcar sintáctico sobre el `while` con `next` y `current`. Nuestra implementación respeta el espíritu de esta equivalencia pero

la materializa en la fase de *generación de código* y no en la fase sintáctica. Hay dos razones para esta decisión:

1. **Preservación de información para diagnóstico:** si el `for` se desazucarara en el parser, los mensajes de error posteriores hablarían de `while`, `next` y `current` en lugares donde el programador escribió `for` y `in`. La información del nodo original se perdería.
2. **Especialización por tipo:** cuando el iterable es un `Range` o un `Vector`, conocemos el tipo concreto y podemos emitir código especializado: aritmética directa de coma flotante para `Range`, indexación entera para `Vector`. La transpilación sintáctica obligaría a usar siempre el despacho indirecto a través del protocolo, incluso cuando es innecesario.

El generador de código para el nodo de bucle `for` detecta si el iterable es un `Range` o un `Vector` y emite uno de dos patrones distintos. Para un `Range`, el patrón emitido es:

```

1  %iter    = call ptr @hulk_range_alloc(double %s, double %e)
2  br label %for_cond
3  for_cond:
4  %has     = call i1 @hulk_range_next(ptr %iter)
5  br i1 %has, label %for_body, label %for_end
6  for_body:
7  %x       = call double @hulk_range_current(ptr %iter)
8  store double %x, ptr %x_slot
9  ; ... <emite el cuerpo, con x visible en el scope> ...
10 br label %for_cond
11 for_end:

```

LISTADO 8.8. IR generado para `for (x in range(s, e))`.

Para un `Vector<T>`, el patrón emitido sustituye `hulk_range_next` y `hulk_range_current` por una comparación directa `i32 idx < size` y una llamada a `hulk_vec_get` con el índice incrementado en cada iteración. Esta especialización elimina dos llamadas a función por iteración a cambio de emitir dos patrones distintos en el generador de código —un compromiso razonable dada la frecuencia de uso de los dos casos.

8.2.4 Vectores como iterables

Los vectores implementan el protocolo `Iterable`, pero en nuestra implementación esta conformidad se realiza de forma *ad hoc* y no a través del mecanismo general de conformidad estructural: el verificador de tipos trata el caso `Vector(T)` especialmente al analizar un `for`, extrayendo el tipo de elemento `T` sin consultar al sistema de protocolos.

La razón es eficiencia. Si los vectores conformaran con `Iterable` a través de la implementación de `next` y `current` como métodos ordinarios, cada iteración pagaría una llamada de método virtual con su despacho de *vtable*, además del coste del propio acceso al elemento. El tratamiento especial permite emitir, para los vectores, el mismo patrón que para los `Range`: comparación directa contra el tamaño y obtención del puntero a la celda con una sola llamada al *runtime*.

8.2.5 Análisis comparativo

El protocolo de iteración con `next/current` (estilo cursor mutante) es una de las dos grandes familias del diseño de iteradores. La otra —usada por Rust, Java desde la versión 8 y

los *stream operators* de C#— es `next() -> Option<T>`: una sola llamada por iteración que combina el avance del cursor y la lectura del elemento, devolviendo `None` cuando no quedan más.

TABLA 8.2. Comparativa de protocolos de iteración.

LENGUAJE	MÉTODOS	MODELO
HULK	<code>next(): Boolean</code> y <code>current(): T</code>	Cursor mutante
Java	<code>hasNext(): bool</code> y <code>next(): T</code>	Cursor mutante (avance en <code>next</code>)
Python	<code>__next__(): T</code> (lanza <code>StopIteration</code>)	Excepción terminadora
Rust	<code>next(): Option<Self::Item></code>	Suma de tipos
C#	<code>MoveNext(): bool</code> y <code>Current: T</code>	Cursor mutante
Go	<code>for k, v := range iterable</code>	Compilador interno
C++	Iteradores con <code>operator++</code> , <code>operator*</code>	Aritmética de punteros

La elección de HULK —dos métodos, `next` avanza y devuelve si queda, `current` lee el valor actual— coincide con la convención de C# (`IEnumerator`). Es ligeramente menos elegante que el modelo Rust de `Option<T>` porque exige dos llamadas por iteración en lugar de una y deja indefinido el comportamiento de `current` cuando `next` ha devuelto `false`. A cambio, no requiere un tipo suma en el sistema de tipos del lenguaje (HULK no tiene tipos suma de primer nivel ni `Option/Result`), por lo que el modelo Rust no es transferible sin antes diseñar e implementar dichos constructores.

8.2.6 Decisiones de diseño

Por qué `current` y no `value`. El verbo elegido para la lectura del elemento es `current`, no `value` o `get`. La razón es semántica: “*current*” sugiere que el iterador tiene un *estado actual* y que la llamada es idempotente; “*value*” o “*get*” sugerirían que cada llamada podría consumir el elemento. Con la regla de que `next` avanza y `current` sólo lee, dos invocaciones consecutivas de `current` sin `next` intercalado retornan el mismo valor.

Por qué `Object` como tipo de retorno declarado en el protocolo. El protocolo declara `current(): Object` y deja al tipo concreto `Range` declarar `current(): Number` (covarianza). La alternativa sería parametrizar el protocolo, `Iterable<T>`, como hace Java con `Iterable<T>`. La razón por la que HULK no lo hace es la ausencia de polimorfismo paramétrico de primer nivel en el lenguaje: no existe la sintaxis `protocol Iterable<T> { ... }`. La parametricidad se recupera por la vía de la covarianza más el tipado nominal puntual: cada tipo iterable redefine `current` con el tipo de elemento concreto y los clientes que necesitan el tipo refinado usan la anotación nominal (`Range` o `Vector<Number>`) en lugar de la genérica (`Number*`).

Por qué el `for` se desazucara en codegen y no en el parser. Discutida en el cuerpo de la subsección anterior: preservación de información de diagnóstico y posibilidad de especialización por tipo concreto del iterable.

8.3 Vectores: del literal a la aloca  n dimensionada

8.3.1 Motivaci  n

Los vectores son colecciones homog  neas indexables y de tama  o fijo, el sustrato sobre el que se construyen la mayor  a de los algoritmos elementales del lenguaje. La forma b  sica del lenguaje admite dos construcciones para crearlos —el literal expl  cito $[e_1, e_2, \dots, e_n]$ y la expresi  n generadora $[\text{expr} \mid x \in \text{iterable}]$ —, anotaci  n de tipo mediante $T[]$ y dos operaciones *built-in*: indexaci  n con $v[i]$ y consulta de tama  o con $v.size()$. Los vectores implementan impl  citamente el protocolo iterable para permitir su uso con el ciclo `for`.

Nuestra implementaci  n cubre las dos formas b  sicas y enriquece el sistema con tres construcciones adicionales:

- **Sintaxis alternativa con llaves:** $\{e_1, e_2, e_3\}$ como equivalente sint  ctico de $[e_1, e_2, e_3]$.
- **Aloca  n dimensionada:** `new T[N]` —reserva un buffer de tama  o N pre-inicializado al cero del tipo— y la variante `new T[N] { i -> e }`, que liga el   ndice del elemento al identificador i y eval  a la expresi  n e en cada posici  n.
- **Vectores bidimensionales:** anotaci  n $T[][]$ y constructor `new T[][][N]`.

Estas construcciones cubren patrones de programaci  n elementales que la sintaxis b  sica maneja con incomodidad: la inicializaci  n de un buffer de N ceros requerir  a escribir `[0 | i in range(0, N)]` con la sobrecarga del `Range` intermedio; la inicializaci  n de una matriz identidad requerir  a un bucle anidado de asignaciones destructivas. M  s cr  tico todav  a, los tests de la suite de validaci  n del compilador exigen estas construcciones; sin ellas, una parte significativa del CI quedar  a sin pasar.

8.3.2 El tipo `Vector<T>` en el sistema de tipos

El sistema de tipos representa los vectores con un constructor param  trico de primer nivel, `HulkType::Vector(Box<HulkType>)`, donde el par  metro es el tipo del elemento. La conformidad de vectores es *invariante* en el par  metro: `Vector<A> ≤ Vector<B>` si y s  lo si $A = B$. Esta elecci  n es la   nica estable en presencia de mutaci  n: un `Vector<Animal>` no puede aceptar un `Vector<Perro>` aunque `Perro` conforme con `Animal` porque escribir un `Animal` en la posici  n de un `Perro` es una violaci  n de tipos en tiempo de ejecuci  n (el problema cl  sico de Java con la covarianza de arreglos, documentado por Bracha y Cook [20]).

8.3.3 Formas de construcci  n

A continuaci  n describimos las cinco formas sint  cticas que el compilador admite para construir un vector, present  ndolas en orden creciente de especificidad.

Literal expl  cito con corchetes: $[e_1, e_2, \dots, e_n]$. La forma b  sica del lenguaje. La gram  tica admite tres variantes: vector vac  o `[]`, vector con elementos `[ArgListNonEmpty]` y la forma generadora descrita m  s abajo. El parser construye un nodo `VectorExpr::Explicit { elements, span }`. El verificador de tipos infiere el tipo del primer elemento como “tipo candidato” del vector y verifica que los dem  s conformen con   l.

Literal explícito con llaves: $\{e_1, e_2, \dots, e_n\}$. Una construcción nuestra —no recogida en la forma básica del lenguaje— pero familiar para el programador con experiencia en C, C++ y Java: la sintaxis de *aggregate initializer*. La producción reduce $\{\text{Expr}, \text{ArgListNonEmpty}\}$ al mismo nodo `VectorExpr::Explicit` que construyen los corchetes. La equivalencia es exacta: el árbol sintáctico abstracto resultante es indistinguible.

La producción *exige una coma inicial obligatoria* (de ahí la forma `Expr, ArgListNonEmpty` en lugar de la natural `ArgListNonEmpty`). La razón es la coexistencia con la producción del bloque de expresiones $\{s_1; s_2; \dots; s_n\}$, que también comienza con una llave seguida de una expresión. Sin la coma obligatoria, el parser LALR(1) sufre un conflicto *shift/reduce* no resoluble con un único token de *lookahead*: tras consumir `{ Expr`, la decisión entre vector y bloque exige inspeccionar el *tercer* símbolo (coma frente a punto y coma), lo que excede la capacidad de LALR(1). Al obligar la coma como tercer símbolo de la producción del vector, la decisión se vuelve local: si tras `Expr` hay una coma, es un vector; si hay un punto y coma o una llave de cierre, es un bloque. El precio que se paga es la inexistencia de las formas $\{x\}$ (unitaria) y $\{\}$ (vacía); el programador que las necesite debe usar corchetes.

Expresión generadora: `[body |  $x \in \text{iterable}$ ]`. La forma para construir un vector cuyo elemento i -ésimo se calcula evaluando `body` con x ligado al i -ésimo elemento del iterable. Es la pieza central de la extensión de vectores y reproduce la notación de construcción de conjuntos de la teoría axiomática: $S = \{f(x) \mid x \in A\}$. La barra vertical `|` coincide con el operador OR lógico, lo que introduce un conflicto *shift/reduce* en la tabla LALR(1) cuando el cuerpo del generador contiene una subexpresión que podría continuar como OR; el conflicto se resuelve con un *lookahead* adicional de dos tokens, inspeccionando si tras `|` viene `IDENTIFIER in`.

El verificador de tipos abre un *scope* con x tipado según el tipo del elemento del iterable (`Number` si el iterable es `Range`; T si es `Vector<T>`) y devuelve `Vector<T'>` donde T' es el tipo del cuerpo.

Aloación dimensionada: `new T[N]`. Una construcción nuestra, exigida por los tests del compilador. Aloca un buffer de N elementos pre-inicializados al *valor por defecto* del tipo T : cero para `Number` y `Boolean`, `null` para tipos puntero (cadenas, objetos, vectores anidados). La gramática extiende el no-terminal `NewExpr` con la producción correspondiente:

```
NewExpr -> 'new' Identifier '[' Expr ']'
```

Nótese que la producción usa `Identifier` directamente y no el no-terminal `TypeName`. La razón es de nuevo LALR(1): si la producción admitiera `TypeName`, el parser tendría que decidir entre dos lecturas al ver `new Number[:` o bien `TypeName(Number[ )` continuado por `N`, o bien `Identifier(Number)` continuado por `[N]`. La forma con `Identifier` evita la ambigüedad localmente.

El parser construye un nodo de la nueva variante `VectorExpr::Alloc { elem_type, size, generator: None, span }`.

Aloación con generador por índice: `new T[N]{ i -> e }`. Refinamiento de la forma anterior: el identificador i se liga al índice de cada elemento (no a un valor extraído de un iterable) y la expresión e se evalúa en cada posición. Es la sintaxis canónica para inicializar

un vector con un patr33n dependiente del 33ndice —tablas de potencias, vectores de muestras de una funci33n, vectores indicadores *one-hot*, filas de una matriz identidad. La producci33n tiene diez s33mbolos:

```
NewExpr -> 'new' Identifier '[' Expr ']' '{' Identifier '->' Expr '}'
```

Construye el mismo nodo `VectorExpr::Alloc` de la forma anterior, con el campo `generator` igual a `Some((var, body))`.

8.3.4 Anotaciones de tipo: $T[]$ y $T[][]$

La sintaxis $T[]$ declara una anotaci33n de tipo *vector de T*, mediante una producci33n simple de tres s33mbolos:

```
TypeName -> Identifier '[' '']
```

El AST lo representa con la variante `TypeName::Vector { name, span }`, y el resolutor de tipos del verificador lo traduce a `Vector<T>`.

La sintaxis $T[][]$ —a33adida por nosotros— extiende la anotaci33n a vectores de dos niveles. La producci33n admite cinco s33mbolos:

```
TypeName -> Identifier '[' ']' '[' '']
```

El AST introduce una variante espec33fica `TypeName::Vector2D { name, span }`, pero el sistema de tipos *no* introduce un nuevo constructor: el resolutor traduce `Vector2D` a la composici33n `Vector<Vector<T>>` del constructor de primer nivel consigo mismo:

```
1 TypeName::Vector2D { name, .. } => {
2     HulkType::Vector(Box::new(
3         HulkType::Vector(Box::new(self.name_to_hulk_type(name)))
4     ))
5 }
```

Esta composicionalidad tiene una consecuencia importante: el verificador de tipos, el generador de c33digo y el *runtime* no requieren ninguna modificaci33n para soportar vectores bidimensionales. El soporte se reduce a una traducci33n de tipos en la fase de an33lisis sint33ctico, y el resto del compilador trata `Vector<Vector<T>>` con el mismo c33digo que trata cualquier `Vector<X>`. El constructor `new T[][N]` reduce a `VectorExpr::Alloc` con `elem_type = Vector(T)` y por tanto genera c33digo id33ntico al de `new (Vector<T>)[N]`: aloca un buffer de N punteros pre-inicializados a `null`, lo que en t33rminos pr33cticos significa *filas no inicializadas*.

El modelo subyacente es, en la terminolog33a habitual de Java y C#, un *jagged array*: un vector de punteros, cada uno apuntando a un vector independiente que puede tener un tama33o distinto. La alternativa —*rectangular array*, un 33nico bloque contiguo accedido con aritm33tica de 33ndices $i \cdot \text{cols} + j$ — ofrecer33a mejor localidad de cach33 para algoritmos densamente num33ricos pero requerir33a un nuevo `HulkType` y un layout de *runtime* separado. Nuestra elecci33n por el modelo *jagged* es consistente con el principio rector del cap33tulo: no introducir nuevos constructores en el sistema de tipos si la operaci33n es expresable como composici33n de los existentes.

8.3.5 Indexación, mutación y el método `size`

La indexación de un vector se reconoce por una producción compartida con la indexación de cualquier expresión postfija:

```
CallOrAccess -> CallOrAccess '[' Expr ']'
```

El verificador exige que la colección sea `Vector<T>` y el índice sea `Number`; devuelve `T` como tipo de la expresión.

La indexación en posición de *lvalue* `v[i] := e` se admite en la asignación destructiva. La rama del generador de código para la indexación en posición de *lvalue* construye el puntero a la celda mediante el accesor del *runtime* y emite un `store` con el valor calculado.

El método `size` se distingue del resto en que el verificador y el generador de código lo *interceptan* en el receptor `Vector<T>` sin pasar por el mecanismo de despacho de protocolos. El verificador reconoce el caso `Vector(_).size()` y devuelve `Number`; el generador de código emite una llamada directa a la función de *runtime* `hulk_vec_size`.

8.3.6 Representación en runtime y verificación de límites

El *runtime* de vectores usa un layout simple: una cabecera de 8 bytes con el número de elementos seguida de las celdas de datos, cada una de 8 bytes —suficiente para almacenar un `f64`, un `i64` o un puntero. La función de asignación usa `alloc_zeroed` para que el buffer venga pre-inicializado al patrón binario cero, que es semánticamente cero para `Number` y `Boolean` y `null` para tipos puntero:

```
1 // Layout: [i64 count][f64/ptr elem0][f64/ptr elem1]...
2 pub extern "C" fn hulk_vec_alloc(count: i32, _element_size: i32) -> *mut
   u8 {
3     let n      = count.max(0) as usize;
4     let layout = Layout::from_size_align(8 + n * 8, 8).unwrap();
5     let ptr    = unsafe { alloc_zeroed(layout) };
6     unsafe { *(ptr as *mut i64) = n as i64; }
7     ptr
8 }
9
10 // Devuelve puntero a la celda i; aborta si i fuera de rango.
11 pub extern "C" fn hulk_vec_get(vec: *mut u8, index: i32, _element_size:
   i32) -> *mut u8 {
12     let size = unsafe { *(vec as *const i64) };
13     if index < 0 || (index as i64) >= size {
14         eprintln!("HULK error en tiempo de ejecucion: indice {} fuera de
           rango [0, {})", index, size);
15         std::process::abort();
16     }
17     unsafe { vec.add(8 + (index as usize) * 8) }
18 }
19
20 pub extern "C" fn hulk_vec_size(vec: *mut u8) -> f64 {
21     unsafe { *(vec as *const i64) as f64 }
22 }
```

LISTADO 8.9. Runtime de vectores: asignación, indexación, tamaño.

La verificación de límites en `hulk_vec_get` es estricta: si el índice está fuera de $[0, \text{count})$, el *runtime* imprime un mensaje descriptivo en `stderr` y aborta el proceso mediante `std::process::abort`. Esta política sigue el principio de Hoare [15]: *una condición que no debería ocurrir no debe producir un valor arbitrario; debe detener la ejecución de la manera más visible posible*.

8.3.7 Reglas de tipado formales

Para las cinco formas de construcción descritas, las reglas de tipado del verificador son las siguientes, donde Γ es el ambiente de tipos y $\vdash$ denota el juicio de tipado:

$$\begin{array}{ll}
\frac{\Gamma \vdash e_1 : T \quad \dots \quad \Gamma \vdash e_n : T}{\Gamma \vdash [e_1, \dots, e_n] : \text{Vector}\langle T \rangle} & \text{(literal explícito, corchetes o llaves)} \\
\frac{\Gamma \vdash e : \text{Range} \quad \Gamma, x : \text{Number} \vdash b : T}{\Gamma \vdash [b \mid x \text{ in } e] : \text{Vector}\langle T \rangle} & \text{(generador sobre rango)} \\
\frac{\Gamma \vdash e : \text{Vector}\langle S \rangle \quad \Gamma, x : S \vdash b : T}{\Gamma \vdash [b \mid x \text{ in } e] : \text{Vector}\langle T \rangle} & \text{(generador sobre vector)} \\
\frac{\Gamma \vdash n : \text{Number}}{\Gamma \vdash \text{new } T[n] : \text{Vector}\langle T \rangle} & \text{(alocación dimensionada)} \\
\frac{\Gamma \vdash n : \text{Number} \quad \Gamma, x : \text{Number} \vdash b : S \quad S \leq T}{\Gamma \vdash \text{new } T[n]\{x \rightarrow b\} : \text{Vector}\langle T \rangle} & \text{(alocación con generador)} \\
\frac{\Gamma \vdash e : \text{Vector}\langle T \rangle \quad \Gamma \vdash i : \text{Number}}{\Gamma \vdash e[i] : T} & \text{(indexación)}
\end{array}$$

8.3.8 Semántica operacional

La semántica operacional adopta una estrategia **ansiosa** (*eager*): todos los elementos se calculan en orden de izquierda a derecha y se almacenan en un buffer recién asignado *antes* de que la expresión completa retorne. Esta elección está alineada con la semántica del resto de HULK —donde los argumentos de funciones, los lados de los operadores binarios y los inicializadores de `let` se evalúan de forma estricta— y se justifica con detalle en la discusión de decisiones de diseño.

Se presenta la semántica en estilo de *gran paso*: el juicio $\sigma \vdash e \Downarrow v, \sigma'$ se lee “en el entorno σ , la expresión e se evalúa al valor v produciendo el entorno σ' ”. Las cinco formas de construcción y la indexación dan lugar a las siguientes reglas:

$$\begin{array}{ll}
\frac{\sigma \vdash e_1 \Downarrow v_1, \sigma_1 \quad \dots \quad \sigma_{n-1} \vdash e_n \Downarrow v_n, \sigma_n}{\sigma \vdash [e_1, \dots, e_n] \Downarrow \text{vec}(v_1, \dots, v_n), \sigma_n} & \text{(E-VecExp)} \\
\frac{\sigma \vdash e \Downarrow \text{vec}(v_1, \dots, v_n), \sigma_0 \quad \forall i. \sigma_0[x \mapsto v_i] \vdash b \Downarrow w_i, \sigma_i}{\sigma \vdash [b \mid x \text{ in } e] \Downarrow \text{vec}(w_1, \dots, w_n), \sigma_n} & \text{(E-VecGen)} \\
\frac{\sigma \vdash n \Downarrow k, \sigma_0 \quad k' = \max(0, \lfloor k \rfloor)}{\sigma \vdash \text{new } T[n] \Downarrow \text{vec}(\underbrace{0_T, \dots, 0_T}_{k' \text{ veces}}, \sigma_0)} & \text{(E-VecAlloc)}
\end{array}$$

$$\frac{\sigma \vdash n \Downarrow k, \sigma_0 \quad k' = \max(0, \lfloor k \rfloor) \quad \forall i \in [0, k'). \sigma_0[x \mapsto i] \vdash b \Downarrow w_i}{\sigma \vdash \text{new } T[n]\{x \rightarrow b\} \Downarrow \text{vec}(w_0, \dots, w_{k'-1}), \sigma_0} \quad (\text{E-VecAllocGen})$$

$$\frac{\sigma \vdash e \Downarrow \text{vec}(v_0, \dots, v_{n-1}), \sigma_0 \quad \sigma_0 \vdash i \Downarrow k, \sigma_1 \quad 0 \leq \lfloor k \rfloor < n}{\sigma \vdash e[i] \Downarrow v_{\lfloor k \rfloor}, \sigma_1} \quad (\text{E-IndexOk})$$

$$\frac{\sigma \vdash e \Downarrow \text{vec}(v_0, \dots, v_{n-1}), \sigma_0 \quad \sigma_0 \vdash i \Downarrow k, \sigma_1 \quad \neg(0 \leq \lfloor k \rfloor < n)}{\sigma \vdash e[i] \Downarrow \perp} \quad (\text{E-IndexErr})$$

La regla E-INDEXERR captura el comportamiento de aborto controlado del *runtime*: un índice fuera de rango no produce un valor sino una transición al estado de error $\perp$, que el *runtime* materializa imprimiendo un mensaje y deteniendo la ejecución. La denotación 0_T en E-VECALLOC es el valor por defecto del tipo T : 0 para Number, false para Boolean, null para tipos puntero.

8.3.9 Generación de código para la forma generadora

La forma generadora —tanto la versión $[b \mid x \text{ in iter}]$ como la versión $\text{new } T[N]\{i \rightarrow b\}$ — requiere un bucle explícito en LLVM IR. La estructura emitida consta de cuatro bloques básicos:

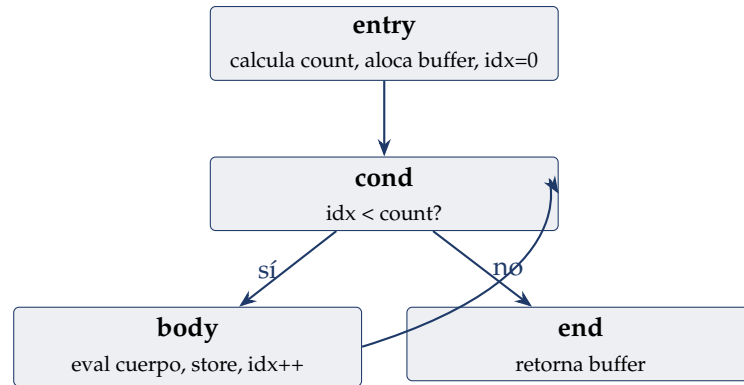


FIGURA 8.1. Grafo de flujo de control del bucle emitido para una expresión generadora o una asignación con generador por índice.

Una sutileza importante surge en el cálculo del count para un generador sobre un Range: $\text{count} = \text{floor}(\text{end} - \text{start})$. Si $\text{start} > \text{end}$, el cálculo produce $\text{count} < 0$, lo que provoca que la instrucción `fptosi` de LLVM genere un *poison value* y el *undefined behavior* subsiguiente. La implementación inserta un *clamp* a $[0, i32::\text{MAX}]$ antes del `fptosi`:

```

1 %diff    = fsub double %end, %start
2 %is_neg  = fcmp ult double %diff, 0.0
3 %nn      = select i1 %is_neg, double 0.0, double %diff
4 %hi      = fcmp ogt double %nn, 2147483647.0
5 %clamp   = select i1 %hi, double 2147483647.0, double %nn
6 %count   = fptosi double %clamp to i32

```

LISTADO 8.10. Clamp del count para evitar poison en `fptosi`.

8.3.10 Análisis comparativo

La familia de construcciones para crear vectores con patrones de inicializaci33n es uno de los puntos del dise13no de un lenguaje en los que m13s divergen las soluciones propuestas. La Cuadro 8.3 resume las sintaxis equivalentes en siete lenguajes mayoritarios para las dos operaciones m13s comunes: “buffer de N ceros” y “buffer de N elementos calculados por una funci33n del 33ndice”.

TABLA 8.3. Sintaxis de construcci33n de vectores en distintos lenguajes.

LENGUAJE	BUFFER EN CERO	CON GENERADOR POR 33NDICE
HULK	<code>new T[N]</code>	<code>new T[N]{ i -> f(i) }</code>
Java	<code>new T[N]</code>	<i>No nativo (loop expl33cito)</i>
Kotlin	<code>IntArray(N)</code>	<code>IntArray(N) { i -> f(i) }</code>
C#	<code>new T[N]</code>	<code>Enumerable.Range(0,N).Select(f).ToArray()</code>
Rust	<code>vec![T::default(); N]</code>	<code>(0..N).map(f).collect()</code>
Python	<code>[0] * N</code>	<code>[f(i) for i in range(N)]</code>
Swift	<code>Array(repeating:0,count:N)</code>	<code>(0..<N).map(f)</code>
C	<code>calloc(N, sizeof(T))</code>	<i>No nativo (loop expl33cito)</i>

La inspiraci33n directa de la sintaxis de HULK es la convenci33n de Kotlin: constructor del vector seguido de un *lambda* sobre el 33ndice. La ventaja sobre el modelo Java/C —en los que el programador debe alocar primero y rellenar despu33s con un bucle— es que `new T[N]{i -> e}` es una *expresi33n*, no una secuencia de *statements*, y por tanto puede aparecer en cualquier posici33n sint13ctica: argumento de funci33n, retorno, inicializador de `let`, brazo de un `if`. La diferencia con Rust —donde la sintaxis `(0..N).map(f).collect()` tambi33n es una expresi33n— es que la versi33n Kotlin/HULK liga el 33ndice mediante una variable expl33cita (i) en lugar de pasarlo a una funci33n an33nima (`|i|` en sintaxis de cierre de Rust), lo que en programas con patrones de inicializaci33n densos hace el c33digo m13s legible.

La forma generadora de HULK —`[b |  $x \in$  iterable]`— pertenece a la familia cl13sica de las *list comprehensions*. Cuadro 8.4 compara la sintaxis y el modo de evaluaci33n.

TABLA 8.4. Comparativa de comprehensions y generators entre lenguajes.

LENGUAJE	SINTAXIS	EVALUACI33N	FILTROS	TIPOS
HULK	<code>[e x in it]</code>	Ansiosa	No	Est13tico inferido
Python	<code>[e for x in it if c]</code>	Ansiosa/Diferida	S33	Din13mico
Haskell	<code>[e x <- xs, c]</code>	Diferida	S33	Est13tico inferido
Scala	<code>for x <- xs yield e</code>	Ansiosa/Diferida	S33	Est13tico inferido
C++20	<code>xs transform(f)</code>	Diferida	S33	Est13tico expl33cito

La elecci33n sint13ctica de HULK —`|` como separador y `in` como palabra reservada, sin filtros,

sin múltiples generadores— es una versión minimalista de las comprehensions de Haskell [7], en cuya teoría unificadora con monads de Wadler [17] encuentran su fundamento.

8.3.11 Decisiones de diseño

Por qué evaluación ansiosa y no perezosa. Tres argumentos sostienen la decisión de evaluación ansiosa. *Primero*, con evaluación ansiosa el tamaño del vector se conoce antes de la asignación, por lo que se requiere una sola llamada a `malloc` con la capacidad exacta. La evaluación perezosa requeriría *thunks* o continuaciones, estructuras que duplicarían la complejidad del *runtime*. *Segundo*, los compiladores de lenguajes perezosos —GHC para Haskell— invierten esfuerzo considerable en evitar el coste de la *thunkificación* con análisis de demanda, análisis de *strictness* y deforestación: un compilador educativo que delega la optimización en LLVM no puede sostener ese coste de desarrollo. *Tercero*, HULK tiene efectos de lado (`print`, asignación destructiva), y la evaluación perezosa en presencia de efectos produce comportamientos sorprendentes porque el orden de evaluación deja de coincidir con el orden textual. La evaluación ansiosa garantiza que los efectos se producen de izquierda a derecha, en consonancia con la semántica del resto del lenguaje.

Por qué | como separador. La barra vertical reproduce la notación matemática $\{f(x) \mid x \in S\}$ y coincide con la convención de Haskell. El hecho de que `|` sea también el operador OR lógico introduce un conflicto *shift/reduce* que se resuelve con *lookahead* de dos tokens. Las alternativas consideradas y descartadas son: `[body for var in iterable]` (más familiar para programadores Python, pero introduce `for` en un contexto distinto al de los bucles), `[body : var in iterable]` (colisiona con la notación de tipo `x : T` ya presente en la gramática) y `[body || var in iterable]` (evita el conflicto pero rompe la correspondencia con la notación matemática).

Por qué vectores inmutables en tamaño (no push/pop). Los vectores no admiten operaciones de `push` o `pop`: una vez creados, su tamaño no cambia. Esta decisión se justifica por la simplicidad del *runtime* (un buffer fijo con cabecera es trivial; vectores dinámicos requerirían estrategias de redimensión exponencial), por la semántica de expresión del lenguaje (todo en HULK es una expresión y preservar la composición funcional `print([x*x | x in range(0,5)]) [3]`) exige que el vector sea valor y no objeto con estado) y por el caso de uso educativo del lenguaje (los programas típicos operan sobre vectores de tamaño conocido).

Por qué sintaxis Kotlin para la aloación con generador. La elección de `new T[N]{ i -> e }` —no `T.tabulate(N, f)` ni `(0..N).map(f)`— prioriza tres criterios: la *expresividad como expresión* (vs. la secuencia de statements de Java); la *variable de índice explícita* (vs. el valor constante de la repetición de Rust `vec![v; n]`); y la *reutilización del operador new* (consistencia con la construcción de objetos sin introducir un verbo adicional).

Por qué el modelo jagged para vectores 2D. La elección por el modelo *jagged* —vector de vectores con un nivel de indirección— y no por el modelo *rectangular* —bloque contiguo con aritmética de índices— responde al principio de minimizar la extensión del sistema de tipos: `T[][]` es expresable como `Vector<Vector<T>` sin introducir un nuevo constructor. El modelo rectangular ofrecería mejor localidad de caché para algoritmos numéricos densos, pero requeriría un `HulkType::Matrix` distinto, un layout de *runtime* separado y reglas de

conformidad específicas. Para los casos de uso esperados de HULK —programas educativos, no cómputo numérico de alto rendimiento— el coste asintótico del modelo *jagged* es comparable y la flexibilidad ganada (filas heterogéneas, redimensión independiente) excede al beneficio del modelo rectangular.

Por qué llaves con coma obligatoria. Discutida en el cuerpo de Apartado 8.3.3: la coma obligatoria $\{ \text{Expr}, \text{ArgListNonEmpty} \}$ resuelve el conflicto *shift/reduce* con la producción del bloque de expresiones $\{ \text{ExprList} \}$ sin recurrir a un parser LALR(k) con $k > 1$. La alternativa —introducir una sintaxis distintiva inequívoca como $\#\{1, 2, 3\}$ de Clojure o $\text{vec}[1, 2, 3]$ de Rust con macros— sería visualmente disruptiva y rompería la expectativa del programador con experiencia en C.

9

Decisiones de diseño significativas

TODO compilador real es una secuencia de decisiones de compromiso entre simplicidad de implementación, eficiencia del código generado y completitud del lenguaje soportado. Este capítulo documenta las decisiones más significativas tomadas durante la construcción del compilador y articula explícitamente las alternativas descartadas, los argumentos a favor de la opción elegida y los costos asumidos.

9.1 LALR(1) artesanal frente a generador externo

El parser está implementado desde cero —gramática como estructura de datos, tablas LALR(1) construidas programáticamente en tiempo de inicialización— en lugar de generarse a partir de YACC, Bison, LALRPOP o ANTLR.

Argumento principal

Transparencia algorítmica. La construcción del autómata LR(0), el cómputo de FIRST y FOLLOW, la propagación de *lookaheads* y la construcción de las tablas ACTION/GOTO constituyen el núcleo técnico de la teoría de *parsing* [1, 3]. Una implementación que delegue ese núcleo en una herramienta externa convierte al *parser* en una caja negra, lo que va en contra del principio de diseño D1 enunciado en la introducción.

Argumento secundario

Inspeccionabilidad y depurabilidad. La gramática es una estructura de datos en memoria que puede imprimirse, recorrerse y analizarse en tiempo de ejecución. Esto resulta determinante a la hora de diagnosticar conflictos *shift/reduce*: el conflicto único de la gramática (asociado al separador | de las expresiones generadoras) pudo identificarse y resolverse inspeccionando la tabla generada, sin necesidad de recompilar el compilador entre iteraciones.

Coste

Implementación más larga (aproximadamente 800 LOC adicionales solo para el módulo LALR) y mayor tiempo de *startup* debido a que las tablas se construyen en cada invocación. Este *overhead* es de pocos milisegundos en programas típicos.

9.2 LLVM como backend vs intérprete vs VM propia

La especificación permite generar código máquina directamente, usar LLVM, o implementar una máquina virtual propia.

Argumento principal

Calidad del código generado. LLVM produce código comparable al de `gcc -O2` para es-

estructuras comunes. Una máquina virtual propia o un intérprete serían varios órdenes de magnitud más lentos para programas con aritmética intensiva.

Argumento secundario

Portabilidad. El mismo IR de LLVM puede compilarse para x86-64, ARM, RISC-V o WebAssembly cambiando un solo parámetro en `TargetMachine`.

Coste

Curva de aprendizaje de LLVM y dependencia de `libLLVM-17.so` en el sistema de *build*. El paquete *inkwell* mitiga la primera dificultad.

9.3 Vtables vs tagged union vs boxed types

Para implementar el polimorfismo dinámico se evaluaron tres alternativas:

1. **Tagged union.** Cada objeto es una union discriminada con un tag de tipo. Simple pero ineficiente: el tamaño de cada objeto es el del tipo mayor de la jerarquía.
2. **Boxed types con dispatch dinámico por nombre.** Cada llamada a método consulta una tabla *hash* por nombre del método. Funciona pero introduce *overheads* de *hashing* y posibles colisiones.
3. **Vtables (elegida).** Cada objeto tiene un puntero a una tabla de funciones. Es la técnica usada por C++, Java JIT, y Rust (para *trait objects*). El costo por llamada es una carga adicional desde cache; el beneficio es el layout estático y conocido.

9.4 Type tags DFS pre-orden vs class objects

El test de tipo `is` se podría implementar de varias formas:

1. **Recorrer la cadena de herencia** en *runtime* comparando nombres o punteros a *class objects*. $O(h)$ donde h es la altura de la jerarquía.
2. **Tabla de class objects.** Almacenar en cada objeto un puntero a su *class*, y consultar una tabla precomputada de relaciones. $O(1)$ con espacio cuadrático en el número de tipos.
3. **Type tags DFS pre-orden (elegida).** $O(1)$ en tiempo, $O(N)$ en espacio.

La elección DFS pre-orden es la combinación óptima tiempo-espacio para el caso común.

9.5 Enlace dinámico vs estático de LLVM

El compilador enlaza dinámicamente con `libLLVM-17.so`. La alternativa (enlace estático) produciría ejecutables de 50–200 MB. El enlace dinámico mantiene el binario en pocos megabytes y comparte la biblioteca con otras herramientas del sistema.

10

Evaluación y pruebas

LA validación del compilador se apoya en dos niveles complementarios: pruebas unitarias escritas en RUST que ejercen cada módulo de forma aislada, y programas HULK completos que recorren el pipeline de extremo a extremo. Esta sección describe la distribución de ambas categorías y el contrato de errores que el compilador expone al usuario.

10.1 Pruebas unitarias por módulo

Las pruebas unitarias residen junto al código que verifican y se ejecutan con cargo `test`. La tabla 10.1 resume su distribución; los totales corresponden al recuento real de funciones marcadas con `#[test]` en el árbol de fuentes.

TABLA 10.1. Distribución de las pruebas unitarias por módulo.

MÓDULO	TESTS	COBERTURA
Lexer	36	Construcción de Thompson, parser de regex, MasterNFA, tokenización con regla del máximo prefijo.
Parser	117	FIRST y FOLLOW, construcción LALR, motor de parseo, gramática completa, traducción de tokens, programas reales.
Semántico	140	Resolución de tipos, conformidad estructural, inferencia en condicionales, protocolos, declaraciones, errores.
Codegen	94	Aritmética, flujo de control, OOP con herencia, despacho dinámico, vectores, rangos y funciones predefinidas.
Total	387	

10.2 Programas HULK de integración

El directorio `tests/hulk/` contiene programas completos que el compilador debe procesar de extremo a extremo. Los programas en `ok/` se compilan, se enlazan con el *runtime* y se ejecutan; su salida estándar se compara contra el archivo `.expected` asociado. Los programas en `errors/` deben fallar en la fase indicada por el subdirectorio y respetar el contrato de errores descrito más abajo.

La suite cubre 101 programas distribuidos como muestra la tabla 10.2.

TABLA 10.2. Programas HULK de integración por categoría.

CATEGORÍA	PROGRAMAS	CONTENIDO
ok/minimal/	20	Aritmética, condicionales, funciones, cadenas, let-in, ciclos básicos.
ok/oop/	10	Tipos con atributos y métodos, herencia, mutación, despacho dinámico, llamadas a base().
ok/types/	10	Anotaciones explícitas, inferencia, conversiones, funciones y constantes predefinidas.
ok/interfaces/	6	Declaración de protocolos, conformidad estructural, parámetros y retornos polimorfos, herencia.
ok/generators/	6	Implementaciones del protocolo Iterable definidas en HULK y consumidas con for.
ok/arrays/	8	Literales con llaves, asignación con new T[N], forma generadora, vectores bidimensionales, mutación, paso por parámetro, size().
ok/extras/	10	Combinaciones de for, while, rangos, ciclos anidados y uso de for dentro de let.
errors/lexical/	6	Caracteres no reconocidos y literales mal formados.
errors/semantic/	15	Variables no declaradas, incompatibilidad de tipos, conformidad rota, aridad incorrecta.
errors/syntactic/	10	Programas sintácticamente mal formados que deben rechazarse en la fase de parseo.
Total	101	

Las categorías ok/interfaces/, ok/generators/ y ok/arrays/ ejercen específicamente las extensiones documentadas en el capítulo 8: protocolos con conformidad estructural, el protocolo Iterable y los vectores con todas sus formas de construcción y anotaciones. Los programas de los subdirectorios ok/lambda/ y ok/macros/ corresponden a funcionalidades que no forman parte de la implementación actual (véase el capítulo 11) y se conservan en el repositorio como material de referencia para trabajo futuro.

10.3 Contrato de errores

El compilador respeta el contrato de interfaz asociado al lenguaje: los errores se reportan por stderr con el prefijo ! y el formato (línea,col) FASE: mensaje; el proceso termina con código de salida distinto de cero. Los programas de errors/ verifican tanto el código de salida como que el mensaje siga este formato, de modo que cualquier regresión en el reporte de errores se detecta de forma automática.

11

Limitaciones conocidas y trabajo futuro

NINGUN compilador es completo. Este capítulo documenta las limitaciones identificadas en la implementación actual y las direcciones de trabajo futuro consideradas.

11.1 Limitaciones actuales

Funciones anónimas (lambdas). La especificación (sección A.13) describe lambdas como funciones de primera clase. La implementación actual solo soporta funciones con nombre. La adición de lambdas requeriría *closures* con captura del entorno léxico, lo que implica un cambio en la representación de los valores funcionales: cada lambda debería representarse como un par (puntero a función, puntero a entorno capturado), con la correspondiente gestión de tiempo de vida sobre el entorno. La adición es mecánica pero no trivial: el código actual del codegen y el del semántico están contruidos bajo el supuesto de que toda función es global, y ese supuesto permea muchas estructuras de datos.

Gestión de memoria. El compilador adopta una estrategia de *solo-asignación*: las cadenas y los objetos se asignan en *heap* con *malloc* y nunca se liberan. Para programas con muchas operaciones de concatenación o asignación de objetos, esto produce un crecimiento monótono del consumo de memoria. Una mejora futura podría ser un *arena allocator* de *regiones* ligado al *let-in* —libera todas las cadenas creadas dentro de un *let* al salir de él—, o un sistema de *reference counting* con detección de ciclos similar al de Python.

Recuperación de errores en el parser. El parser se detiene en el primer error sintáctico. Un parser con recuperación podría reiniciar en el siguiente punto de sincronización (`;` o `}`) y reportar todos los errores en una sola pasada. La técnica clásica de Burke y Fisher [14] es directamente aplicable a la tabla generada por el constructor LALR(1) sin necesidad de regenerar la gramática.

Spans en errores de codegen. Los errores internos del generador de código no incluyen la posición en el fuente. Las fases anteriores propagan *Span* en todos los nodos del AST; propagarlos hasta el codegen mejoraría la experiencia de diagnóstico en caso de errores internos del compilador.

Vectores sin mutación estructural. Los vectores son de tamaño fijo: una vez creados, no admiten *push* ni *pop*. Agregar mutación estructural requeriría sustituir el layout actual (un puntero al buffer con cabecera de tamaño) por una estructura de *capacity-and-length* similar al `Vec<T>` de RUST, con una estrategia de redimensión exponencial para amortizar el coste.

11.2 Trabajo futuro

- **Closures y lambdas:** representar funciones de primera clase con un par (*puntero a función, puntero a entorno*).
- **Generadores perezosos:** variantes lazy de las expresiones generadoras para permitir colecciones conceptualmente infinitas.
- **Inferencia Hindley-Milner completa:** el sistema actual infiere tipos a través de expresiones simples pero no resuelve constraints polimórficos complejos. H-M permitiría funciones genéricas como `function map(v: [A], f: A -> B): [B]`.
- **Optimizaciones específicas de HULK:** *inlining* de métodos monomorficos, análisis de escape para objetos de corta vida (sustituyendo *heap* por *stack*).
- **Backend WebAssembly:** cambiar el TargetMachine de LLVM para emitir WebAssembly permitiría ejecutar programas HULK en el navegador.

12

Conclusiones

EL compilador descrito en este reporte implementa el lenguaje HULK de extremo a extremo: a partir de un fichero fuente `.hulk`, produce un binario ELF x86-64 que el sistema operativo carga y ejecuta sin más dependencias que `libc`, `libm` y la biblioteca de *runtime* aportada por la propia distribución. Las características nucleares del lenguaje —expresiones aritméticas y de cadena, control de flujo expresivo, funciones de primer orden, tipos con herencia simple, protocolos con conformidad estructural variante y operadores `is/as` de prueba y conversión de tipos— quedan implementadas en su totalidad, junto al sistema de iterables y la extensión propia de vectores con expresiones generadoras desarrollada en el Capítulo 8.

Tres decisiones técnicas estructuran la implementación. La primera —construir el *frontend* a mano en lugar de generarlo a partir de una herramienta externa— privilegia la transparencia del autómata LALR(1) y permite resolver en el constructor de la tabla un conflicto deliberado que ninguna herramienta genérica acomoda con elegancia (el separador `|` de las expresiones generadoras, idéntico al operador OR lógico). La segunda —asignar etiquetas de tipo en orden DFS pre-orden sobre la jerarquía de herencia— reduce el operador `is` a una comparación entera de dos cláusulas, una mejora que sitúa el coste del despacho de tipos por debajo del de cualquier comprobación implementada como recorrido de tabla. La tercera —resolver el despacho a través de protocolos mediante un *switch* sobre el `type_tag`, con ramas terminadas en llamadas estáticas— contrasta con el mecanismo habitual en C++ y Rust (tablas adicionales por interfaz) y aprovecha el hecho de que el verificador semántico conoce en tiempo de compilación el conjunto completo de tipos conformes a cada protocolo invocado.

La extensión de vectores con expresiones generadoras sitúa a HULK en la familia de lenguajes que ofrecen soporte de primera clase para transformaciones declarativas de colecciones, junto a Python, Haskell, Scala y, más recientemente, C++20. Las diferencias respecto a esos cuatro lenguajes —y documentadas con detalle en la Cuadro 8.4— no son tanto de expresividad como de *integración*: la extensión convive con el sistema de tipos estático del lenguaje, con los protocolos estructurales y con la jerarquía de objetos sin introducir un subsistema paralelo. El precio asumido ha sido renunciar a filtros, a generadores múltiples y a evaluación perezosa; el beneficio, un *runtime* de pocos cientos de bytes y un IR LLVM que el *pass manager* optimiza con la misma eficacia que un bucle aritmético equivalente escrito a mano. Cada uno de esos compromisos está justificado por separado en la sección de decisiones de diseño de Apartado 8.3.

Las direcciones de trabajo enumeradas en el Capítulo 11 delimitan las extensiones naturales del compilador en su estado actual: la introducción de funciones de primera clase con

cierres léxicos, la recuperación de errores sintácticos para reportar múltiples errores en una sola compilación, y la sustitución del esquema actual de asignación monotónica por un *allocator* basado en regiones ligadas al ámbito de las expresiones `let`. Ninguna de ellas es conceptualmente difícil; cada una representa una ampliación localizada del compilador construido.

Bibliografía

- [1] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley, 2006.
- [2] K. L. Thompson. “Programming Techniques: Regular Expression Search Algorithm.” *Communications of the ACM*, 11(6):419–422, 1968.
- [3] F. L. DeRemer. *Practical Translators for LR(k) Languages*. PhD thesis, MIT, 1969.
- [4] C. Lattner, V. Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation.” *Proceedings of the International Symposium on Code Generation and Optimization*, 2004.
- [5] B. Stroustrup. *The Design and Evolution of C++*. Addison-Wesley, 1994.
- [6] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [7] S. Peyton Jones (ed.). *Haskell 98 Language and Libraries: The Revised Report*. Cambridge University Press, 2003.
- [8] M. Odersky, L. Spoon, B. Venners. *Programming in Scala*. Artima Press, 2008.
- [9] S. Klabnik, C. Nichols. *The Rust Programming Language*. No Starch Press, 2019.
- [10] inkwell: Safe Rust bindings for LLVM. <https://github.com/TheDan64/inkwell>.
- [11] R. Bringhurst. *The Elements of Typographic Style* (3rd ed.). Hartley & Marks, 2004.
- [12] A. Miede. *A Classic Thesis Style: An Homage to The Elements of Typographic Style*. LaTeX template, 2011.
- [13] A. W. Appel. *Modern Compiler Implementation in ML*. Cambridge University Press, 1998.
- [14] K. D. Cooper, L. Torczon. *Engineering a Compiler* (2nd ed.). Morgan Kaufmann, 2011.
- [15] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice Hall International, 1985.
- [16] R. Nystrom. *Crafting Interpreters*. Genever Benning, 2021. <https://craftinginterpreters.com/>
- [17] P. Wadler. “Comprehending Monads”. *Proceedings of the 1990 ACM Conference on LISP and Functional Programming*, pp. 61–78, 1990.
- [18] N. Wirth. “The Design of a Pascal Compiler”. *Software: Practice and Experience*, 1(4):309–333, 1971.
- [19] B. H. Liskov, J. M. Wing. “A Behavioral Notion of Subtyping”. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994.
- [20] G. Bracha, W. Cook. “Mixin-Based Inheritance”. *Proceedings of OOPSLA/ECOOP '90*, pp. 303–311, 1990.